

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ 사이킷런 라이브러리 소개

□ kNN

□ 회귀분석

□ 나이브 베이즈

□ 의사결정나무

□ 서포트벡터머신

AI/BigData 인텐시브 과정

Section 1. 라이브러리 소개

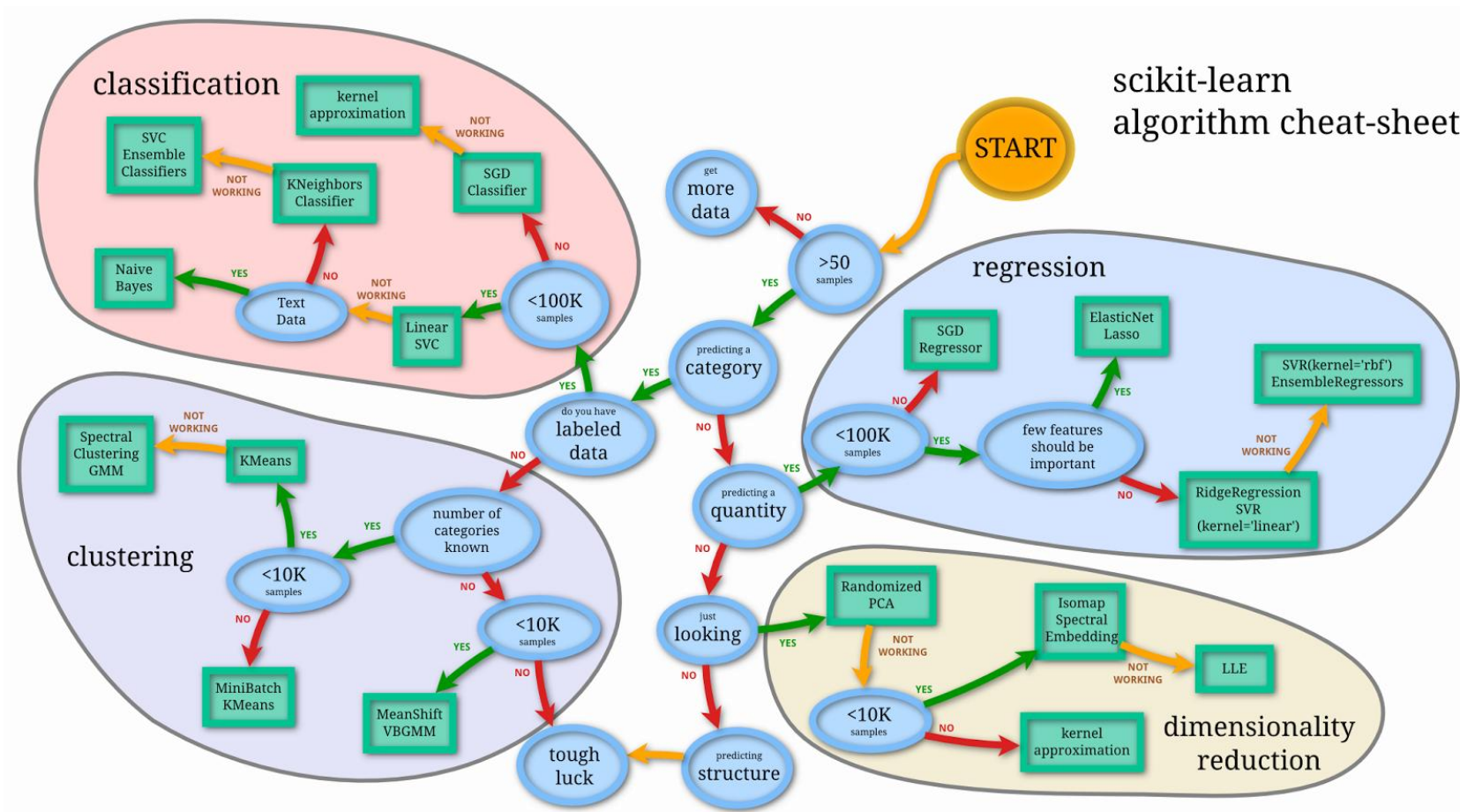
Section 1-1. 사이킷런 라이브러리

사이킷런(scikit-learn)

파이썬을 활용해 머신러닝을 사용할 수 있게 도와주는 라이브러리



사이킷런(scikit-learn)



사이킷런(scikit-learn)

알고리즘 별로 편리하게 사용할 수 있도록 제공

메소드	기능
fit	모델 적합
predict	예측
score	모델 성능 평가

AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-1. k 최근접 이웃의 기초 개념

k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)

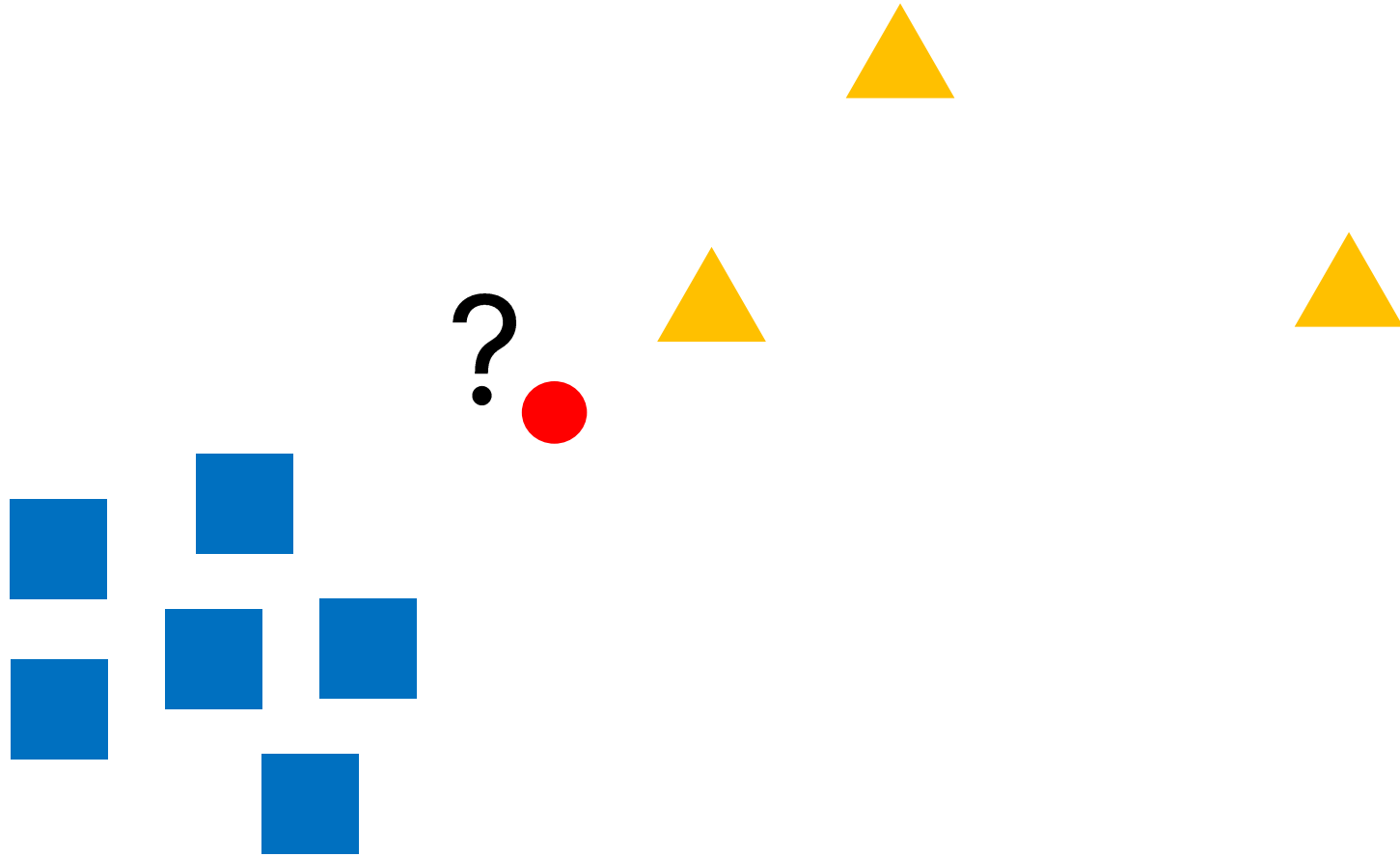
K Nearest Neighbor

K

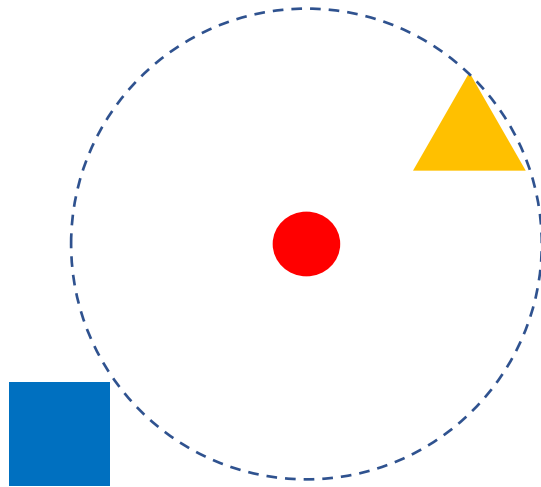
최근접

이웃

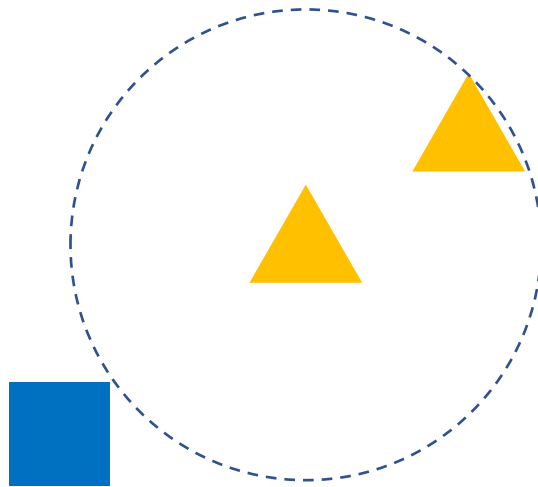
k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



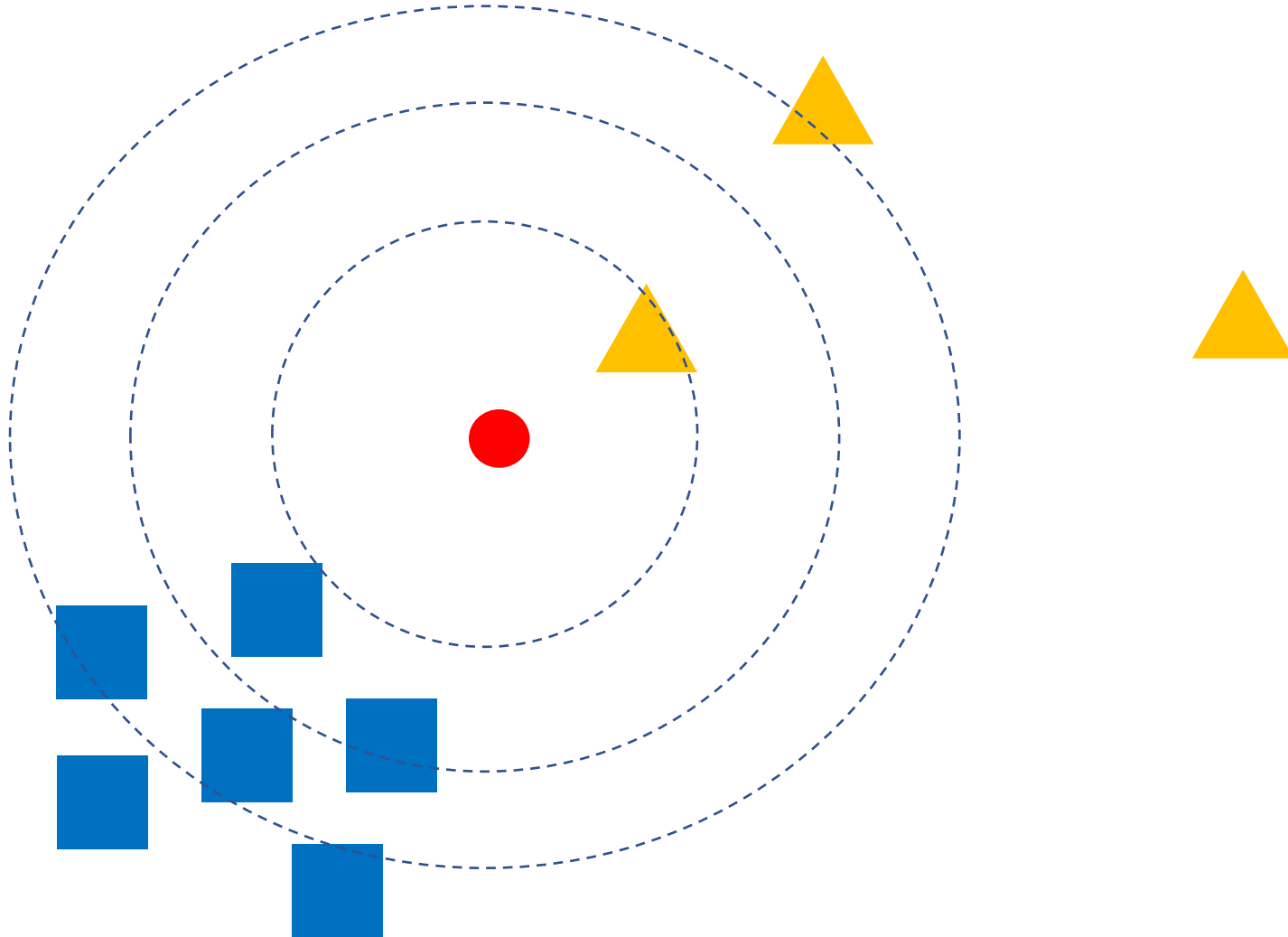
k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



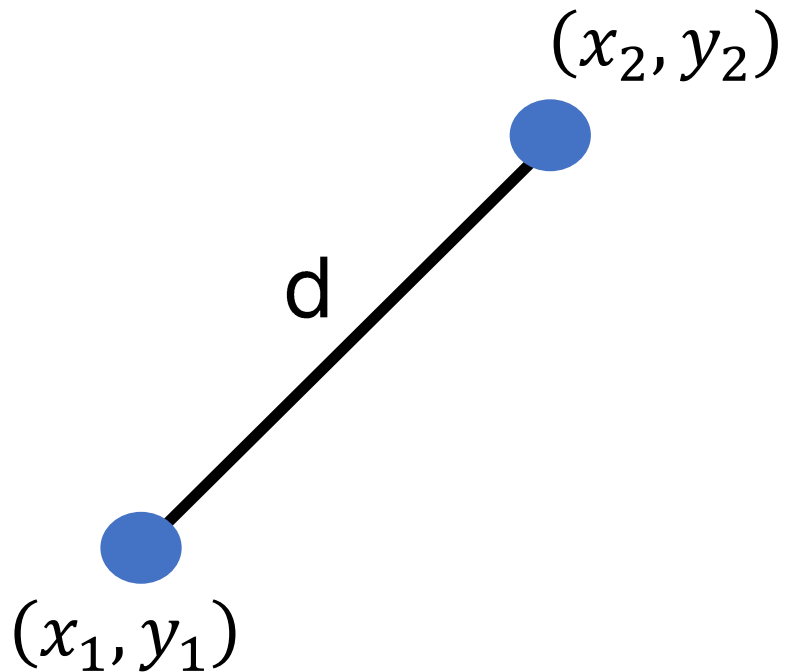
AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-2. 유클리디언 거리

유클리디언 거리

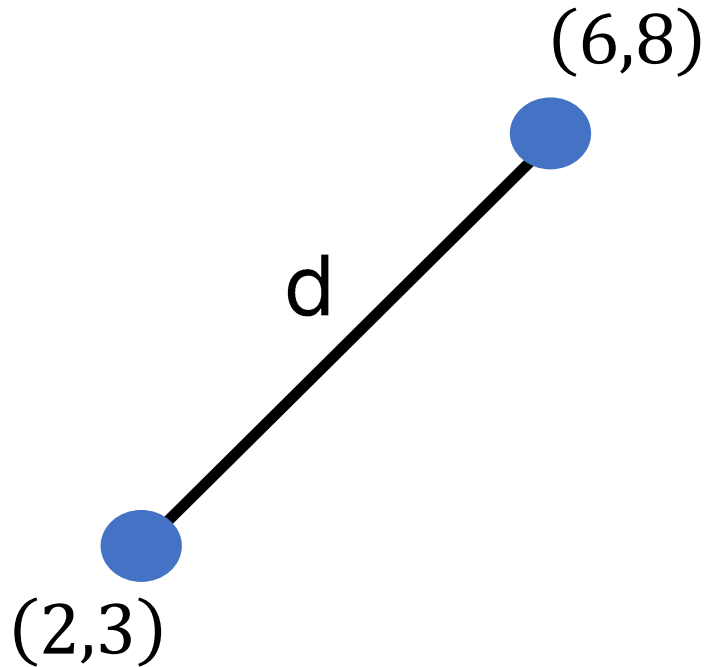
데이터 포인트 간 거리 측정 기준은 기본적으로 유클리디언 거리



유클리디언 거리

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

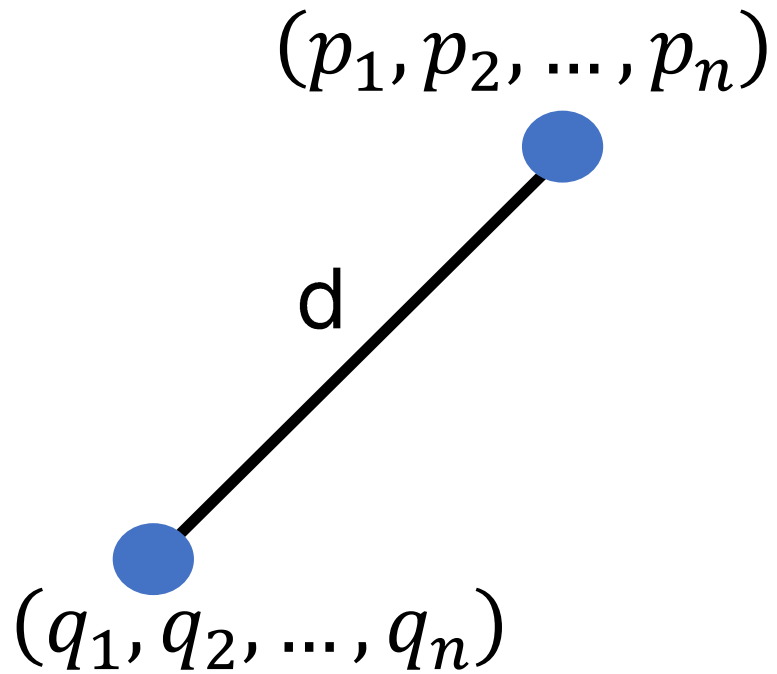
유클리디언 거리(2차원)



유클리디언 거리

$$\begin{aligned} d &= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \\ &= \sqrt{(6 - 2)^2 + (8 - 3)^2} \\ &= \sqrt{16 + 25} = \sqrt{41} \\ &= 6.4 \end{aligned}$$

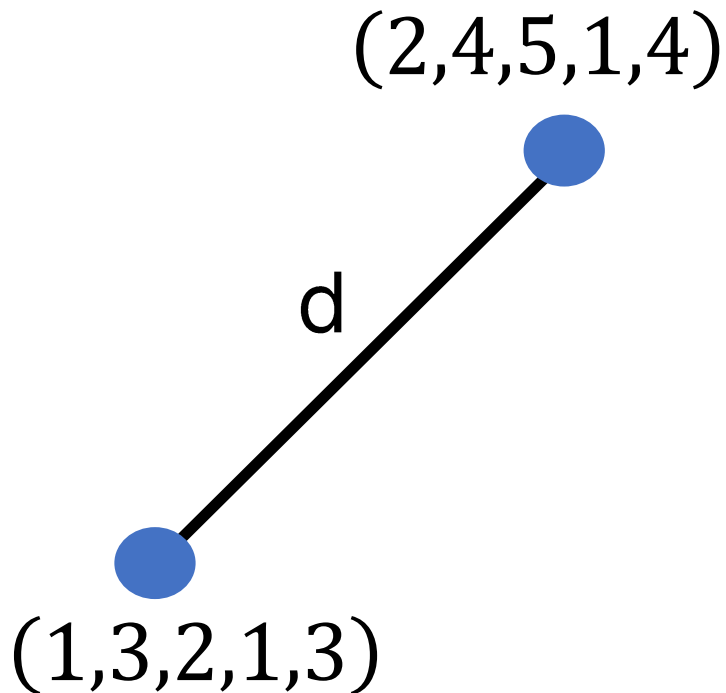
유클리디언 거리(n차원)



유클리디언 거리

$$d = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

유클리디언 거리(n차원)



유클리디언 거리

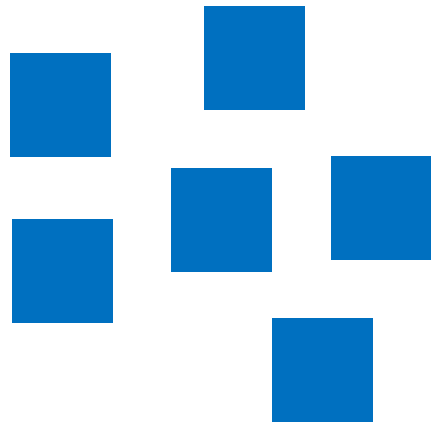
$$\begin{aligned} d &= \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2} \\ &= \sqrt{(2 - 1)^2 + (4 - 3)^2 + (5 - 2)^2 + (1 - 1)^2 + (4 - 3)^2} \\ &= \sqrt{1^2 + 1^2 + 3^2 + 0^2 + 1^2} \\ &= \sqrt{12} \\ &= 3.46 \end{aligned}$$

AI/BigData 인텐시브 과정

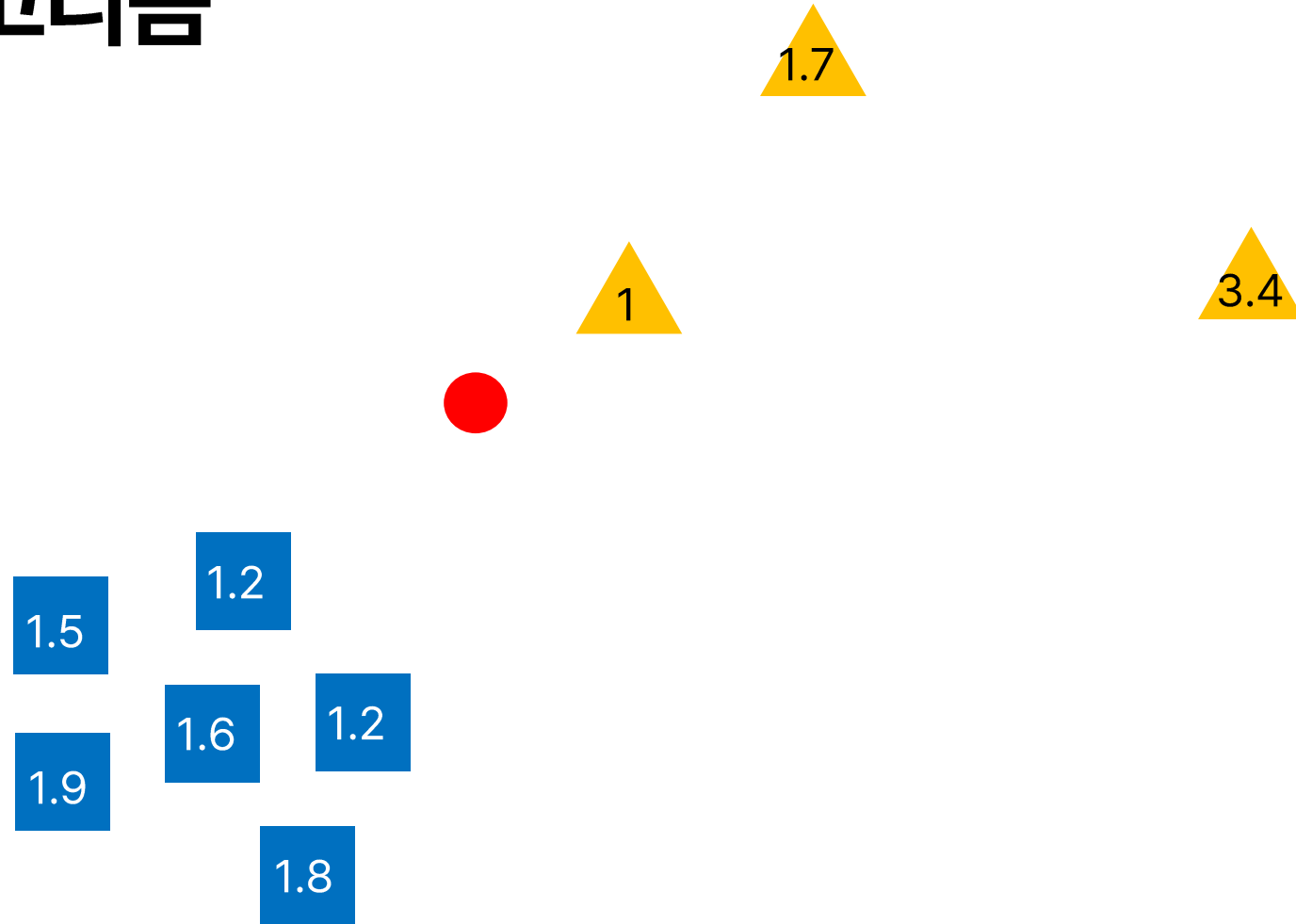
Section 2. K 최근접 이웃

Section 2-3. k 최근접 이웃 알고리즘

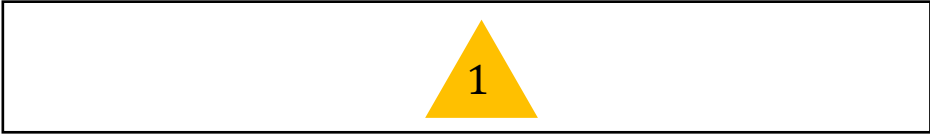
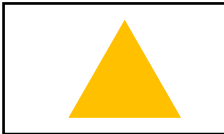
k 최근접 이웃 알고리즘



k 최근접 이웃 알고리즘



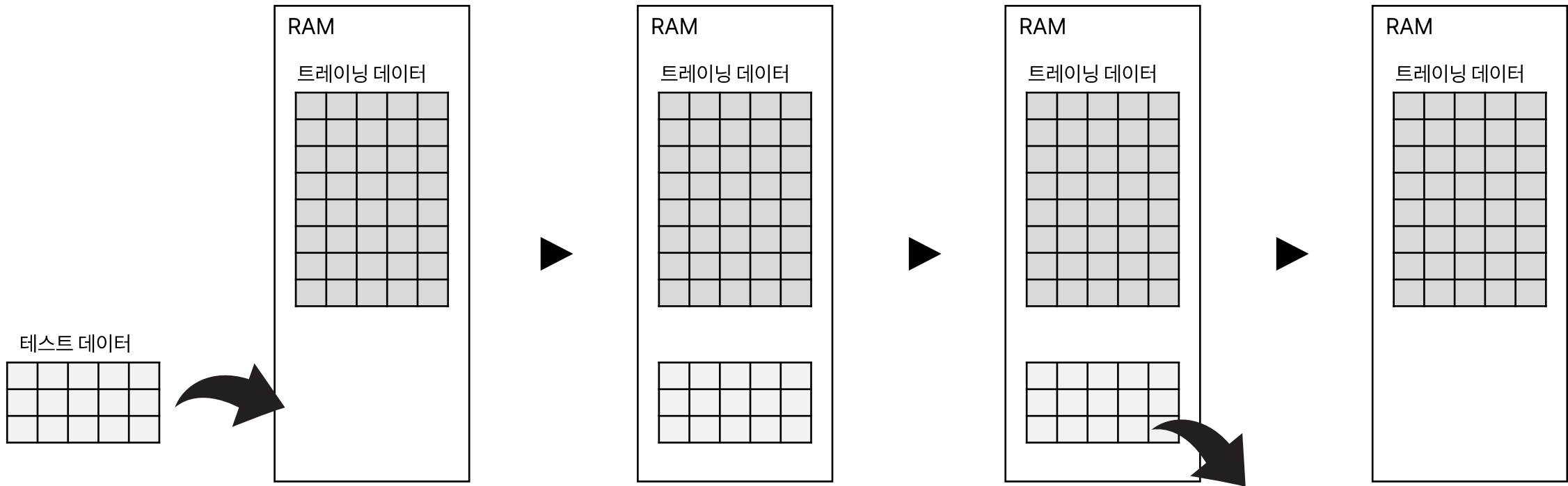
k 최근접 이웃 알고리즘

1-Nearest Neighbor		
3-Nearest Neighbor		
5-Nearest Neighbor		
7-Nearest Neighbor		

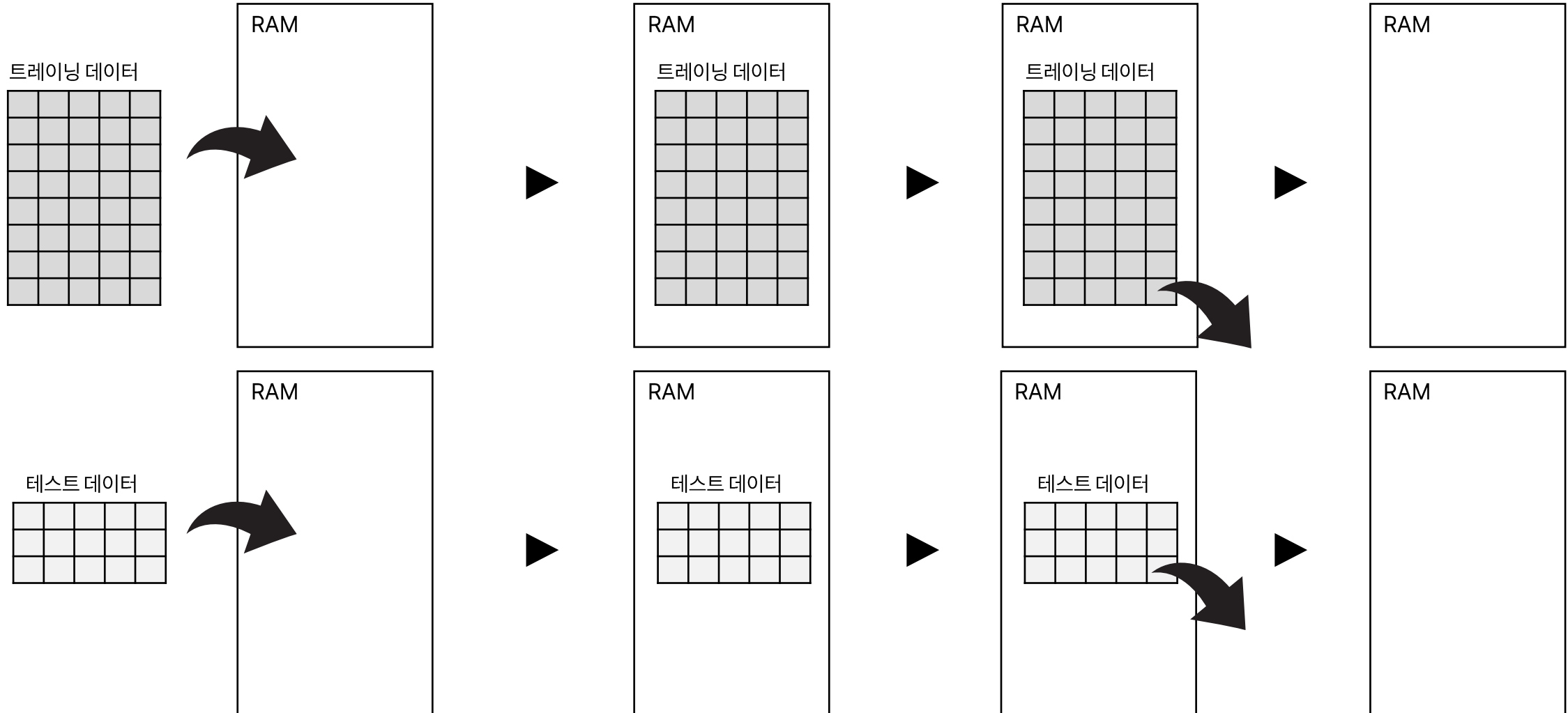
k 최근접 이웃 알고리즘

- ✓ 가장 가까이 있는 데이터 포인트 라벨에 속한다고 보는 방법
- ✓ 가까이 있는 데이터 1개를 보면 1-최근접 이웃
- ✓ 가까이 있는 데이터 n개를 보면 n-최근접 이웃

게으른 학습(lazy learning)



열정적 학습(eager learning)



AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-4. k 최근접 이웃 분류

k 최근접 이웃 알고리즘

```
[1]: import pandas as pd  
df = pd.read_csv("../data/iris.csv")
```

```
[2]: df
```

```
[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

k 최근접 이웃 알고리즘

```
[3]: df['class'].unique()
```

```
[3]: array([0, 1, 2], dtype=int64)
```

```
[4]: features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

X = df[features]
y = df['class']
```

```
[5]: X
```

```
[5]:
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

k 최근접 이웃 알고리즘

```
In [6]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [7]: ▶ 1 X_tn
```

Out[7]:

	sepal_length	sepal_width	petal_length	petal_width
61	5.9	3.0	4.2	1.5
92	5.8	2.6	4.0	1.2
112	6.8	3.0	5.5	2.1
2	4.7	3.2	1.3	0.2
141	6.9	3.1	5.1	2.3
...	...	...	...	...
9	4.9	3.1	1.5	0.1
103	6.3	2.9	5.6	1.8
67	5.8	2.7	4.1	1.0
117	7.7	3.8	6.7	2.2
47	4.6	3.2	1.4	0.2

112 rows x 4 columns

k 최근접 이웃 알고리즘

```
•[8]: #Standardization
      from sklearn.preprocessing import StandardScaler
      std_scale = StandardScaler()
      std_scale.fit(X_tn)
      X_tn_std = std_scale.transform(X_tn)
      X_te_std = std_scale.transform(X_te)
```

```
[9]: X_tn_std
```

```
[-8.01118523e-01],
[-5.60984968e-01,  1.49274053e+00, -1.28103155e+00,
 -1.31530348e+00],
[-1.02212490e+00, -2.42210516e+00, -1.65358660e-01,
 -2.86933568e-01],
[ 7.07149859e-01, -1.19254753e-01,  9.50314227e-01,
  7.41436341e-01],
[ 9.37719827e-01,  5.71600368e-01,  1.06188152e+00,
  1.64126001e+00],
[ 1.30724937e-01, -1.96153508e+00,  6.71396005e-01,
  3.55797625e-01],
[ 9.37719827e-01, -1.27067995e+00,  1.11766516e+00,
  7.41436341e-01],
[-3.30414999e-01, -1.27067995e+00,  5.77759173e-02,
 -1.58387330e-01],
[ 2.09056967e+00, -1.19254753e-01,  1.28501609e+00,
  1.38416753e+00],
[ 4.76579890e-01,  5.71600368e-01,  5.04045072e-01,
```

k 최근접 이웃 알고리즘

```
[10]: # 학습
      from sklearn.neighbors import KNeighborsClassifier
      clf_knn = KNeighborsClassifier(n_neighbors=2)
      clf_knn.fit(X_tn_std, y_tn)
```

```
[10]: ▼ KNeighborsClassifier ⓘ ?
      KNeighborsClassifier(n_neighbors=2)
```

```
[11]: # 예측
      knn_pred = clf_knn.predict(X_te_std)
      print(knn_pred)
```

```
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 2]
```

k 최근접 이웃 알고리즘

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More ▾](#)[Prev](#)[Up](#)[Next](#)

scikit-learn 1.3.1

[Other versions](#)

Please [cite us](#) if you use the software.

[sklearn.neighbors.KNeighborsClassifier](#)[KNeighborsClassifier](#)[Examples using](#)[sklearn.neighbors.KNeighborsClass:](#)

sklearn.neighbors.KNeighborsClassifier

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None)
```

[\[source\]](#)

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters:

n_neighbors : int, default=5

Number of neighbors to use by default for [kneighbors](#) queries.

weights : {'uniform', 'distance'}, callable or None, default='uniform'

Weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. In this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

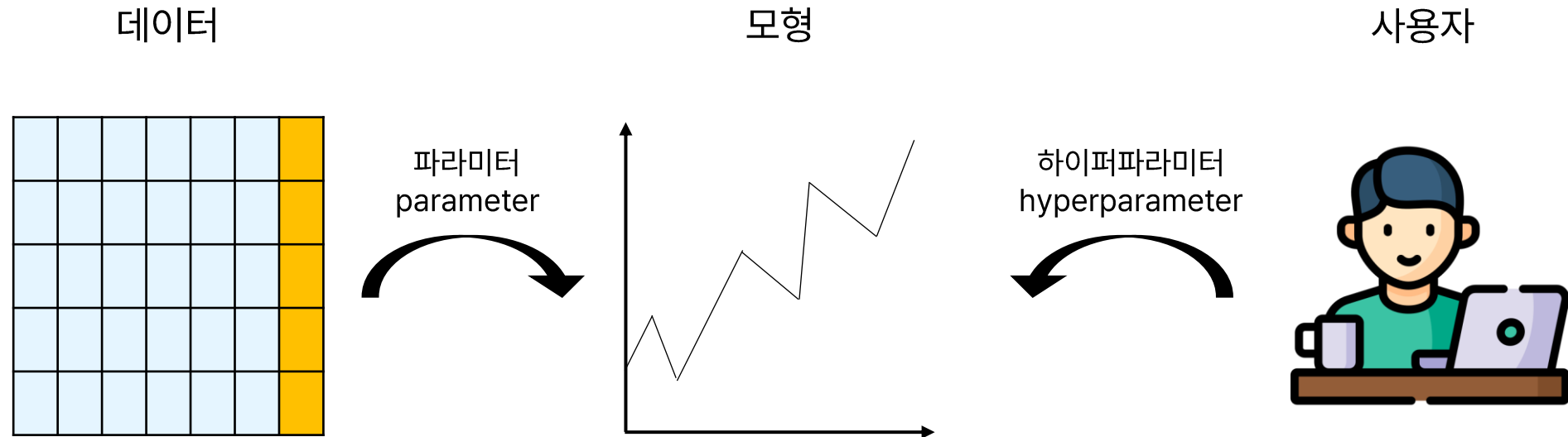
Refer to the example entitled [Nearest Neighbors Classification](#) showing the impact of the `weights` parameter on the decision boundary.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use [BallTree](#)
- 'kd_tree' will use [KDTree](#)
- 'brute' will use a brute-force search.

파라미터(parameter) vs 하이퍼 파라미터(hyper parameter)



k 최근접 이웃 알고리즘

```
•[12]: # 정확도(accuracy)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_te, knn_pred)
print(accuracy)
```

0.9473684210526315

```
[13]: # 정밀도(precision)
from sklearn.metrics import precision_score
precision = precision_score(y_te, knn_pred, average='macro')
print(precision)
```

0.9421296296296297

```
[14]: # 리콜
from sklearn.metrics import recall_score
recall = recall_score(y_te, knn_pred, average='macro')
print(recall)
```

0.9421296296296297

```
[15]: # F1 스코어
from sklearn.metrics import f1_score
f1 = f1_score(y_te, knn_pred, average='macro')
print(f1)
```

0.9421296296296297

k 최근접 이웃 알고리즘

```
[16]: # confusion matrix 확인
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_te, knn_pred)
print(conf_matrix)
```

실제

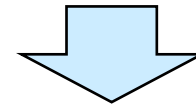
예측

```
[[13  0  0]
 [ 0 15  1]
 [ 0  1  8]]
```

```
[17]: # 분류 레포트 확인
from sklearn.metrics import classification_report
class_report = classification_report(y_te, knn_pred)
print(class_report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	13
1	0.94	0.94	0.94	16
2	0.89	0.89	0.89	9
accuracy			0.95	38
macro avg	0.94	0.94	0.94	38
weighted avg	0.95	0.95	0.95	38

		예측		
		0	1	2
실제	0	13	0	0
	1	0	15	1
	2	0	1	8



		예측	
		0	not 0
실제	0	13	0+0
	not 0	0+0	15+1+1+8=25

AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

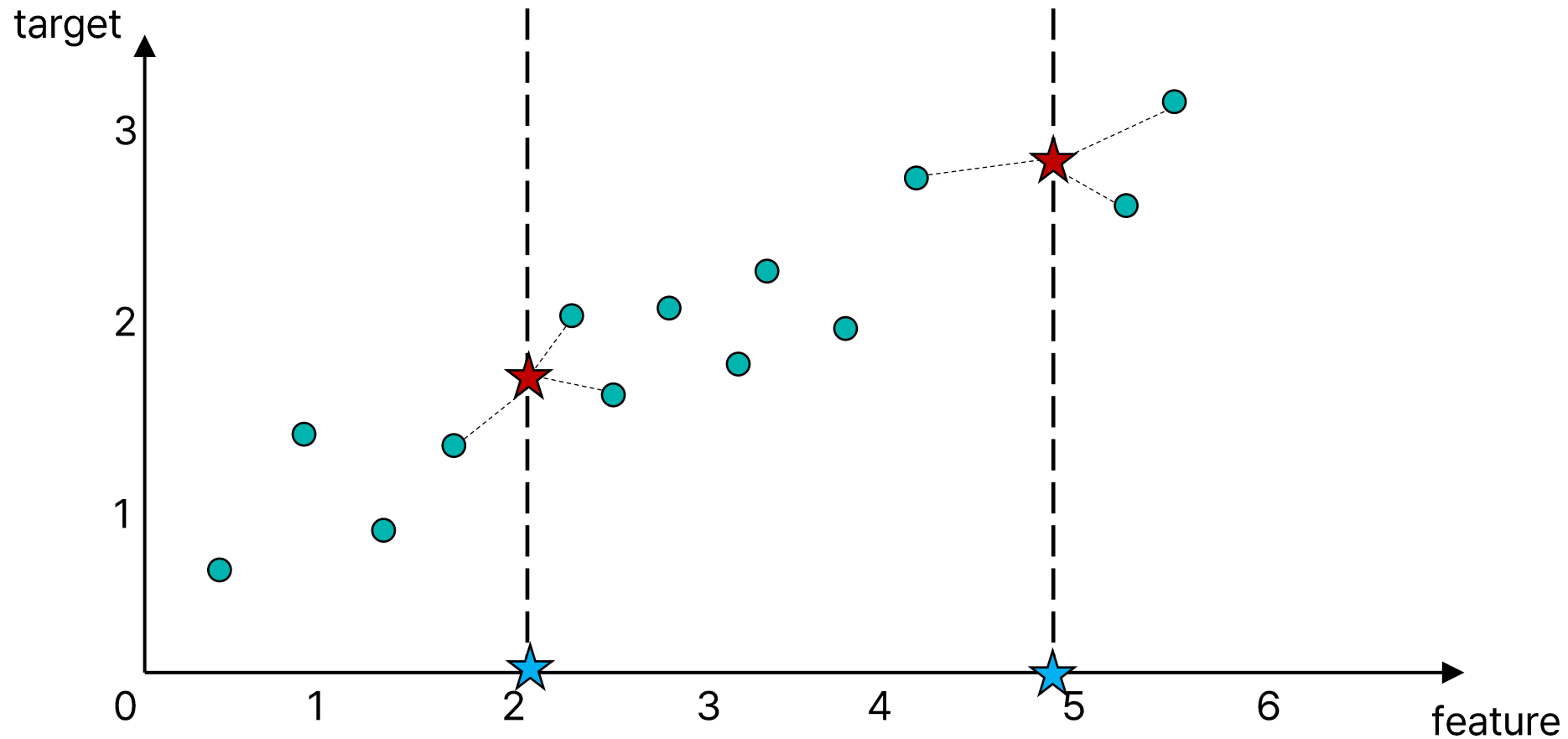
Section 2-5. k 최근접 이웃 회귀

k 최근접 이웃 알고리즘

● training data/target

★ test prediction

★ test data



k 최근접 이웃 알고리즘

```
[18]: import pandas as pd
df = pd.read_csv("../data/house_prices.csv")
```

```
[19]: df
```

[19]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

k 최근접 이웃 알고리즘

```
[20]: df['MEDV'].unique()
```



```
[20]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
        21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
        15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
        24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
        25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
        17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
        22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
        20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
        14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
        15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
        19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
        32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
        42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
        31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
        44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
        25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
        28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
        10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
        12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
        7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
        10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

k최근접이웃회귀

```
[21]: df.columns

[21]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
         'PTRATIO', 'B', 'LSTAT'],
         dtype='object')

[22]: features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']

X = df[features]
y = df['MEDV']

[23]: X

[23]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

k 최근접 이웃 알고리즘

```
[24]: # 트레이닝/테스트 데이터 분할
      from sklearn.model_selection import train_test_split
      X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
[25]: #데이터 표준화
      from sklearn.preprocessing import StandardScaler
      std_scale = StandardScaler()
      std_scale.fit(X_tn)
      X_tn_std = std_scale.transform(X_tn)
      X_te_std = std_scale.transform(X_te)
```

k 최근접 이웃 알고리즘

```
[26]: # 학습
from sklearn.neighbors import KNeighborsRegressor
clf_knn = KNeighborsRegressor(n_neighbors=2)
clf_knn.fit(X_tn_std, y_tn)
```

```
[26]: KNeighborsRegressor
KNeighborsRegressor(n_neighbors=2)
```

```
[27]: # 예측
knn_pred = clf_knn.predict(X_te_std)
print(knn_pred)
```

```
[22.  17.6  23.3  10.35 20.55 19.3  20.9  18.75 23.15 14.15  7.95 10.8
 13.6  11.05 38.5  26.8  22.2  32.45 25.8  22.55 24.05 20.05 19.7  28.4
 21.85 21.9  16.55 17.6  44.9  18.55 14.4  18.6  19.9  21.35 27.35 20.25
 11.8  36.35 15.1  16.05 25.15 21.85 20.25 19.95 21.65 24.3  24.4  14.35
 16.2  23.75 21.15 23.15 23.3  27.9  17.9  20.7  19.55 17.45 13.6  19.65
 22.5  21.45 30.35 24.8  20.25 33.45 14.6  18.9  12.9  24.85 21.95 20.55
 23.3  30.45 27.4   8.7  39.65 24.05 29.1  17.15 22.1  18.6  16.75 39.65
 39.65 24.8  25.8  12.6  20.55 14.3  19.15 10.65 22.7  22.25 22.2  23.9
 12.   23.8  13.85 18.6  25.   19.75 26.3  23.1  22.1  18.9  12.3  17.95
 22.2  22.85 36.15 15.55 18.9  18.8  16.65 21.55  7.95 19.4  10.75 39.6
 36.7  12.05 19.55 19.3  20.5  20.95 32.15]
```

k 최근접 이웃 알고리즘

```
[28]: # 모형 평가-MSE  
      from sklearn.metrics import mean_squared_error  
      mse = mean_squared_error(y_te, knn_pred)  
      print(mse)
```

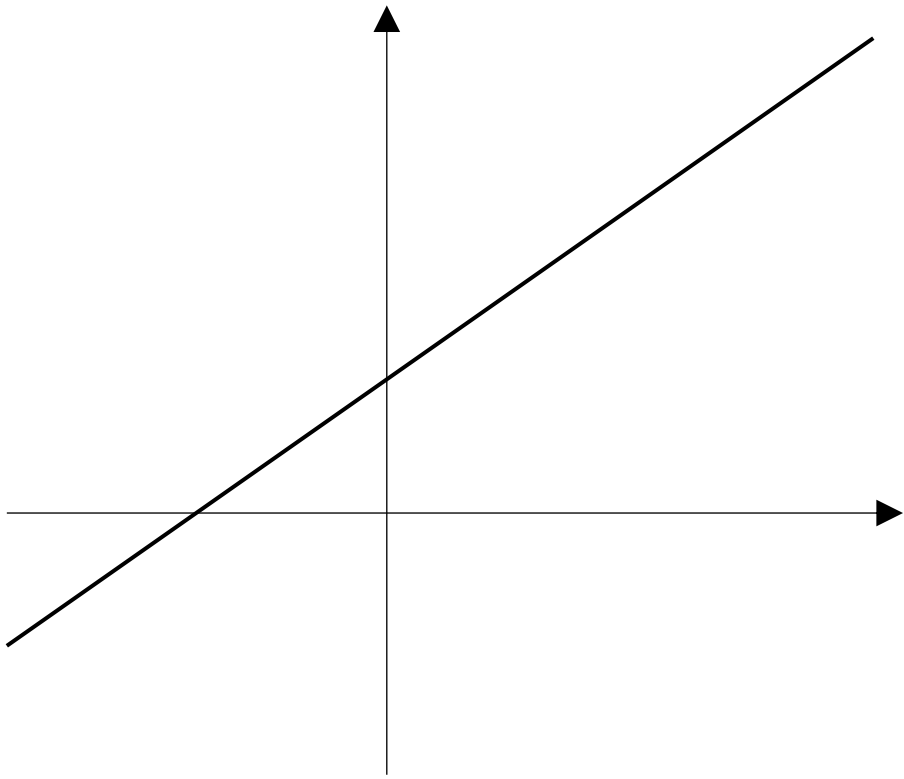
```
26.36383858267716
```

AI/BigData 인텐시브 과정

Section 3. 선형 회귀 분석

Section 3-1. 선형 회귀 분석의 개념

선형 회귀 분석

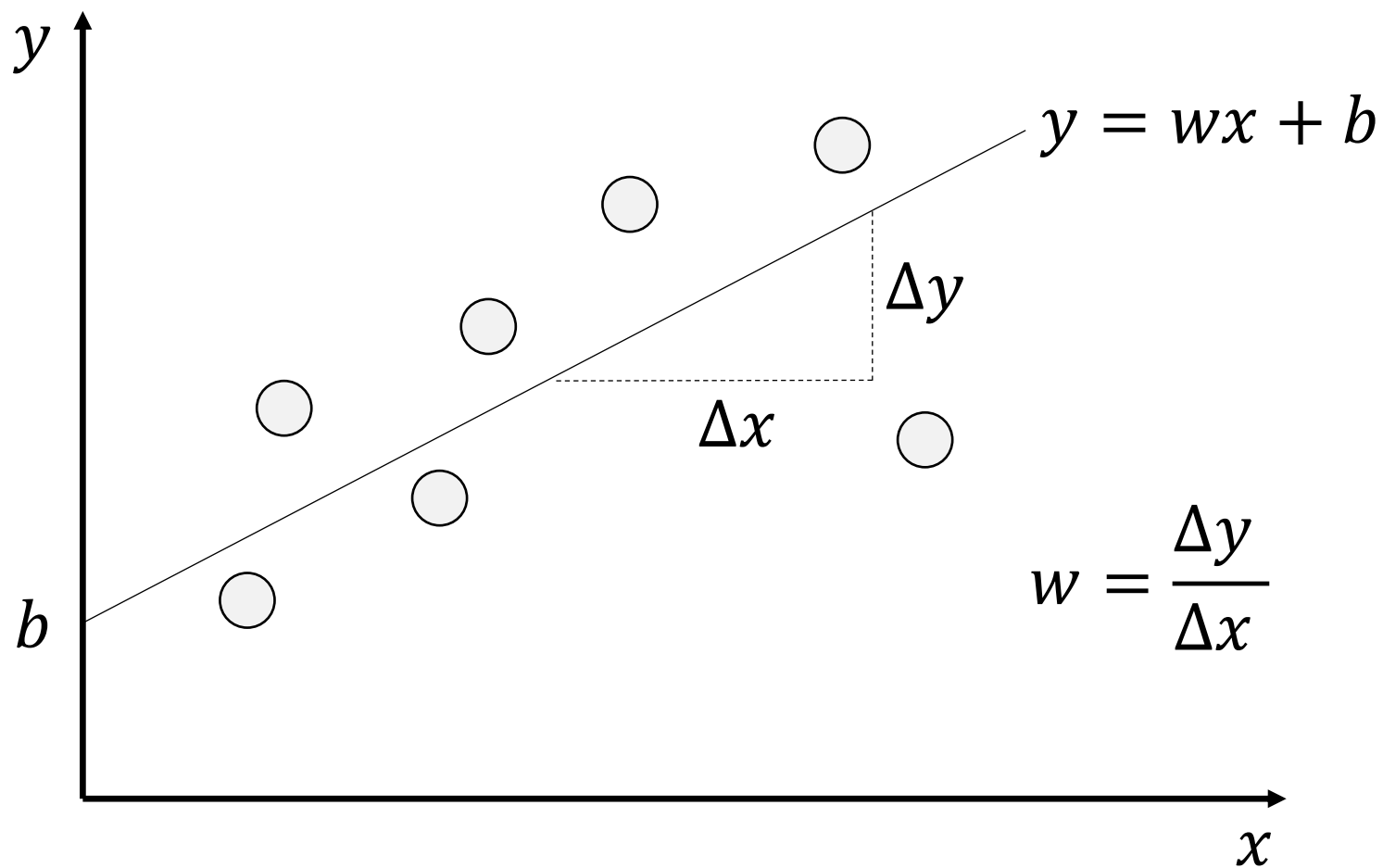


$$y = ax + b$$

기울기

y절편

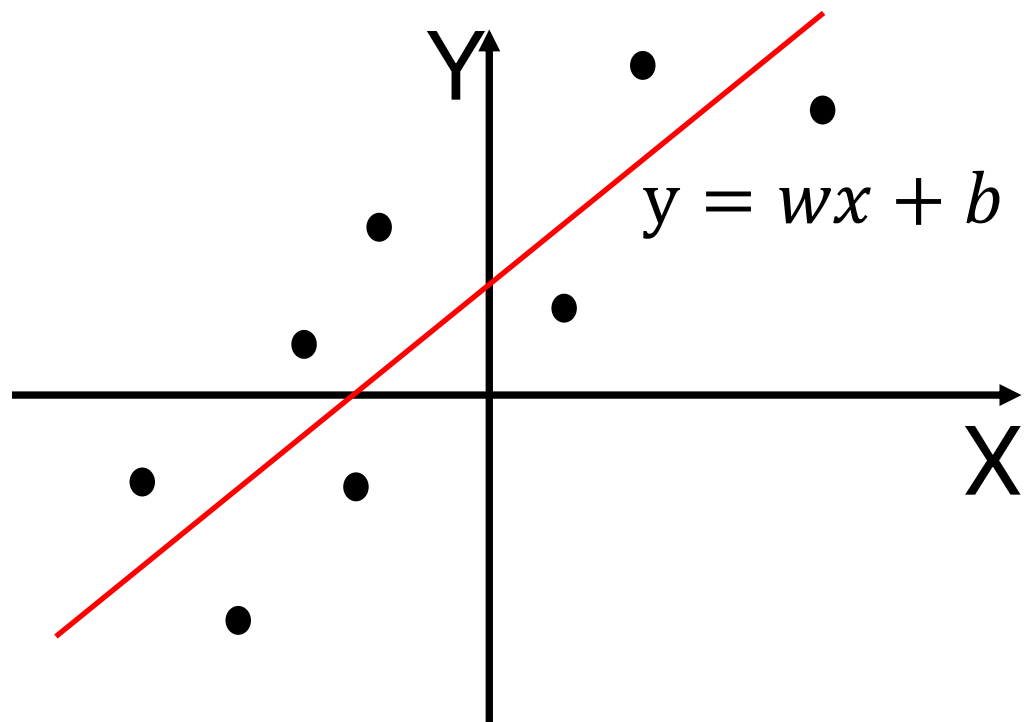
선형 회귀 분석



$$y = wx + b$$

기울기 y절편

선형 회귀 분석



선형 회귀 분석

$$f(\mathbf{x}_i) = \hat{y}_i = w_1x_1 + w_2x_2 + \cdots + w_px_p + b$$

$$f(\mathbf{x}_i) = \hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

선형 회귀 분석

첫번째 피처에 대한 가중치

첫번째 피처

스칼라

스칼라

p번째 피처

$$f(\mathbf{x}_i) = \hat{y}_i = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b$$

$$f(\mathbf{x}_i) = \hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{pmatrix} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix}$$

$$\mathbf{w}^T \mathbf{x} = (w_1 \quad w_2 \quad \cdots \quad w_p) \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix}$$

w를 구하는 방법

$$y_i = f(\mathbf{x}_i) + e_i, \quad i = 1, 2, \dots, n$$

실제값

모형

오차

$$\text{오차 제곱합} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n [y_i - f(\mathbf{x}_i)]^2 = \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

선형 회귀 분석

X

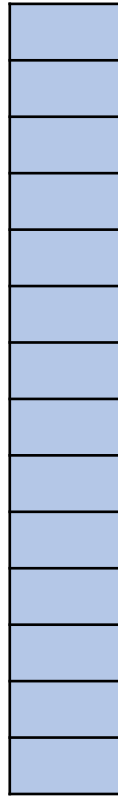
독립 변수, 피처

y

종속 변수, 타겟

선형 회귀 분석

X



독립 변수, 피처

y



종속 변수, 타겟

회귀 분석의 가정

(1) 변수 x 와 y 사이에는 다음과 같은 관련성이 존재한다.

$$E(y) = \beta_1 x + \beta_0$$

$$E(y) = wx + b$$

(2) 피쳐 x 값이 주어졌을 때, 타겟 y 는 정규 분포를 따른다.

(3) 이 때, y 의 평균은 $E(y) = wx + b$ 로 피쳐 x 값에 따라 변하나 분산은 x 값에 관계 없이 일정하다.

(4) 피쳐 x 는 오차 없이 측정할 수 있는 수학 변수(mathematical variable)이며, 타겟 y 는 측정 오차를 수반하는 확률 변수(random variable)이다.

(5) 타겟 y 의 오차들은 서로 독립이다.

회귀 분석의 가정(수식화)

$$y_i = wx + b + \epsilon_i$$

$$y_i \sim N(wx_i + b, \sigma^2)$$

$$\epsilon_i \sim N(0, \sigma^2)$$

$$\text{Cov}(\epsilon_i, \epsilon_j) = 0$$

$$\epsilon_i = y_i - \hat{y} = y_i - (wx + b)$$

$$\text{Var}(y_i) = \sigma^2 \quad \text{등분산 가정}$$

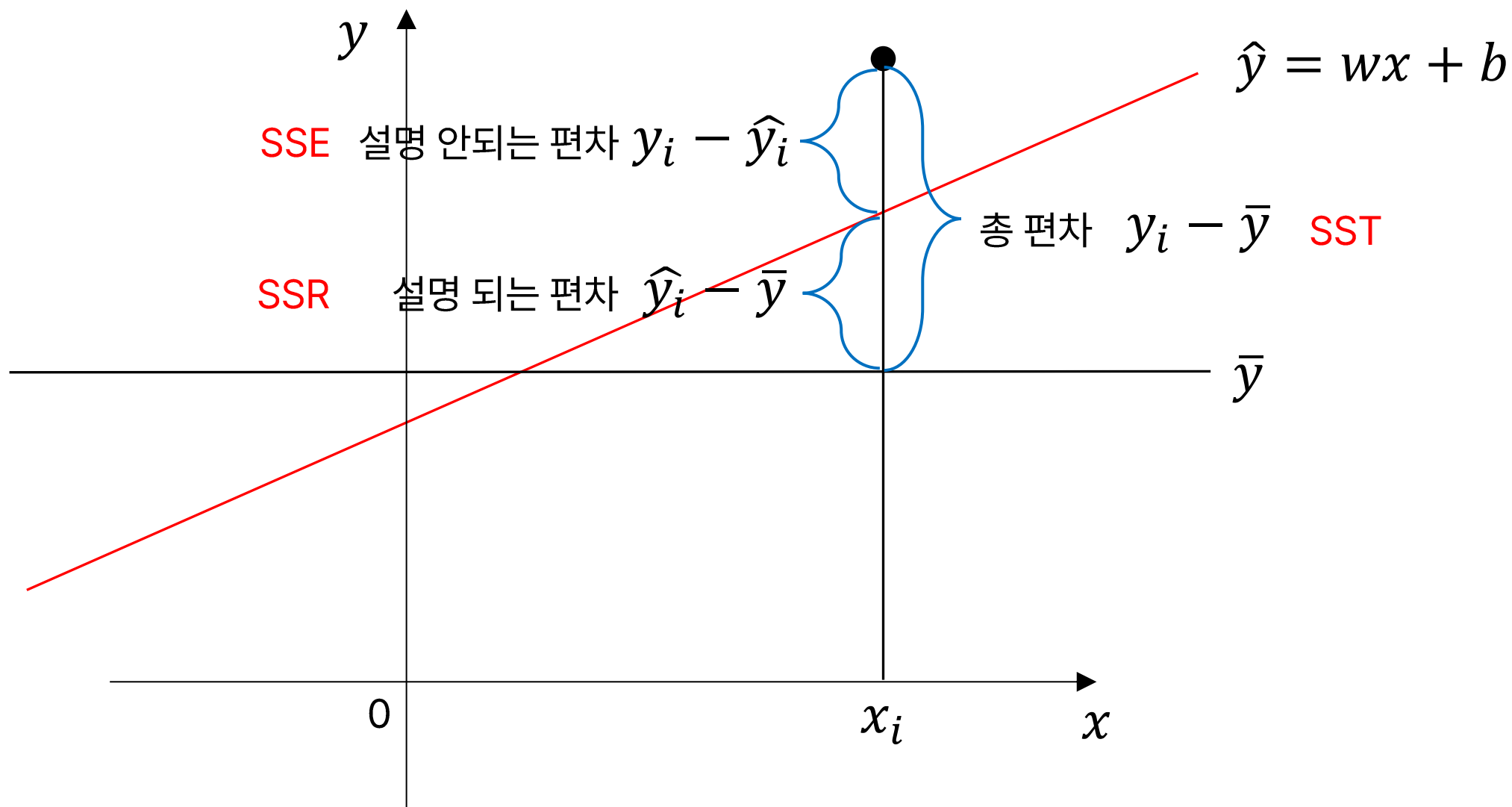
$$\text{Var}(\epsilon_i) = \sigma^2 \quad \text{등분산 가정}$$

최소 제곱 추정량(least square estimator)

$$S = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - wx - b)^2$$

오차 제곱합을 최소로 하는 w, b 구하기

선형 회귀 분석



선형 회귀 분석

$$SST = SSE + SSR$$

총 변동

회귀 모형에 의해

회귀 모형에 의해

설명 안되는 변동

설명 되는 변동

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

결정 계수

$$r^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$0 \leq r^2 \leq 1$$

이 범위라고 흔히 알려져 있지만 음수값도 나올 수 있음

전체 변동 중 회귀 모형으로 설명가능한 변동의 비율

결정 계수

$$r^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$-\infty \leq r^2 \leq 1$$

어떻게 음수 값이 나올까요? 예를 들어 봅시다.

전체 변동 중 회귀 모형으로 설명가능한 변동의 비율

r 제곱값이 음수가 나오는 경우

$$R^2 = 1 - \frac{SSE}{SST}$$

y	$\hat{y}$
1	5
2	4
3	3
4	2
5	1

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2 = (1 - 3)^2 + (2 - 3)^2 + (3 - 3)^2 + (4 - 3)^2 + (5 - 3)^2$$

$$= 4 + 1 + 0 + 1 + 4 = 10$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (1 - 5)^2 + (2 - 4)^2 + (3 - 3)^2 + (4 - 2)^2 + (5 - 1)^2$$

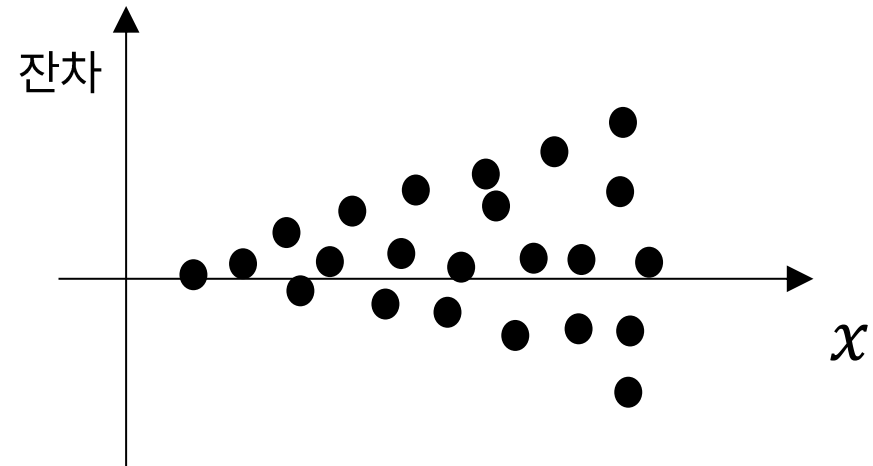
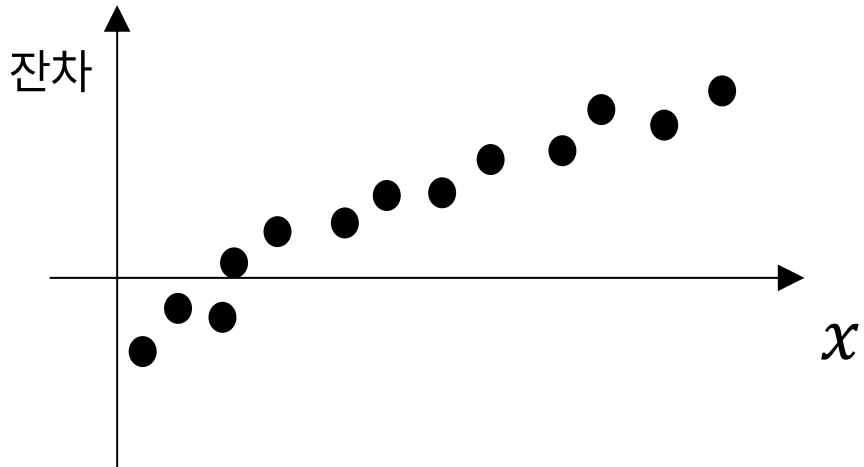
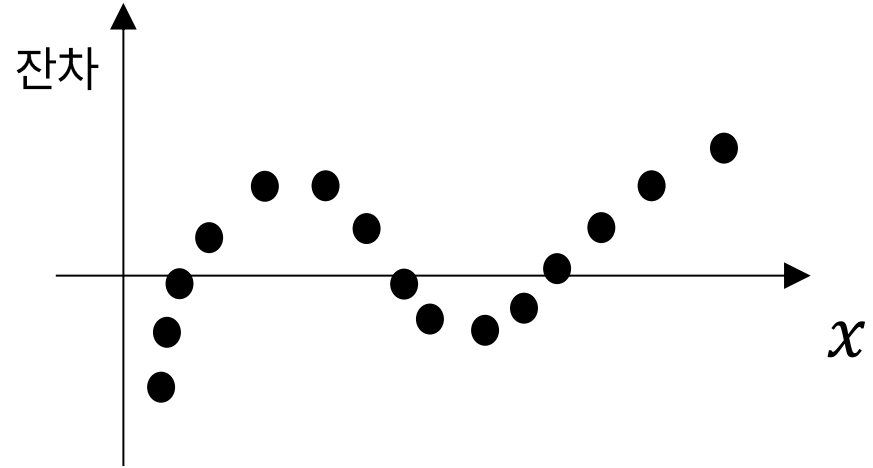
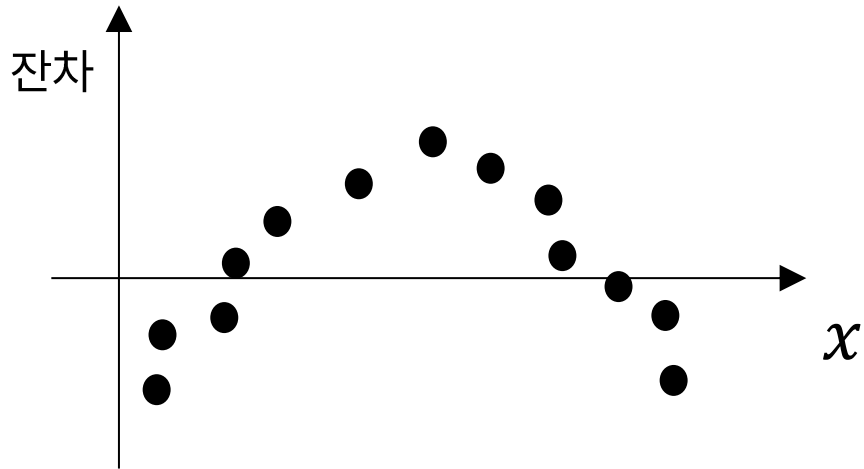
$$= 16 + 4 + 0 + 4 + 16 = 40$$

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{40}{10} = 1 - 4 = -3$$

평균으로 예측하는 것보다도 못할 때

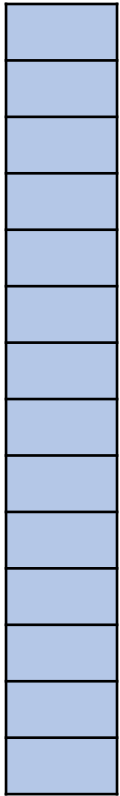
$$\bar{y} = 3$$

가정을 만족하지 않는 경우



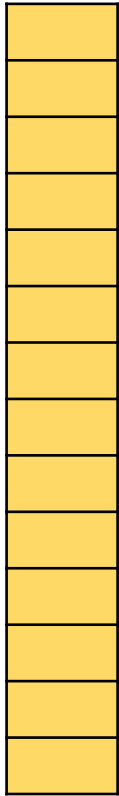
단순 회귀 분석의 개념

X



독립 변수, 피처

y



종속 변수, 타겟

$$y_i = wx + b + \epsilon_i$$

단순 회귀 분석

$$\mathbf{X} = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w \\ b \end{pmatrix}$$

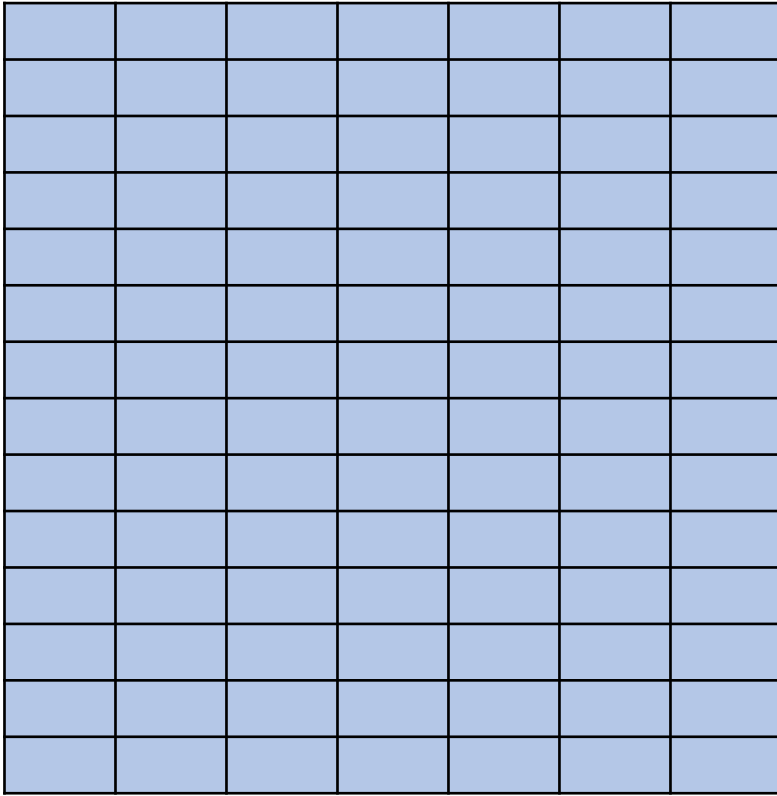
$$\mathbf{y} = \mathbf{X}\mathbf{w}$$

단순 회귀 분석

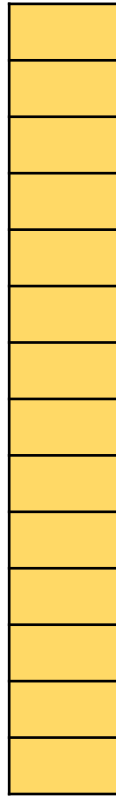
$$\mathbf{X} = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_1 \\ \beta_0 \end{pmatrix}$$

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

선형 회귀 분석

 X 

독립 변수, 피쳐

 y 

종속 변수, 타겟

$$y = Xw + \epsilon$$

선형 회귀 분석

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} & 1 \\ x_{21} & x_{22} & \cdots & x_{2p} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_n^T & 1 \end{pmatrix}$$

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \\ b \end{pmatrix}$$

$$\mathbf{y} = \mathbf{X}\mathbf{w}$$

선형 회귀 분석

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$\frac{\partial \mathbf{e}}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}) = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w} \equiv \mathbf{0}$$

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\mathbf{e} = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$= (\mathbf{y}^T - \mathbf{w}^T \mathbf{X}^T) (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$= \mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

다중공선성(multicollinearity)

 X

독립 변수, 피쳐

어떤 독립 변수가 다른 독립 변수들의 선형 조합으로 표현될 수 있는 경우

$$\text{EX) } x_1 = x_2 + x_3$$

차원 축소

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$



노트북
판매량



디자인



전체 성능



cpu 성능

차원 축소

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$



노트북
판매량

디자인

전체 성능

cpu 성능

$$Corr(x_1, x_2)$$

$$Corr(x_1, x_3)$$

$$Corr(x_2, x_3)$$

카이제곱 검정

Section



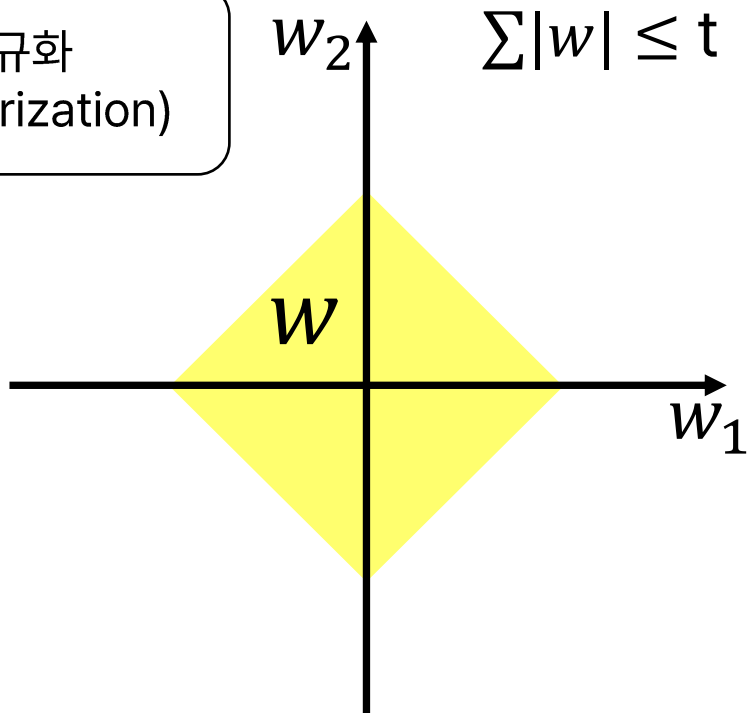
[illegible]

Regularization

$$y = w_1x_1 + w_2x_2 + b$$

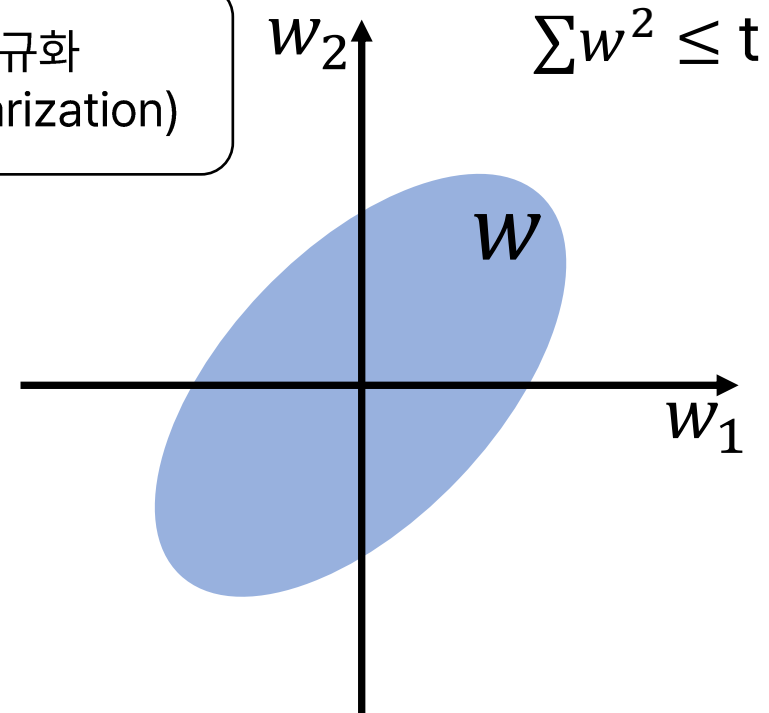
Lasso Regression

L1 정규화
(L1 regularization)

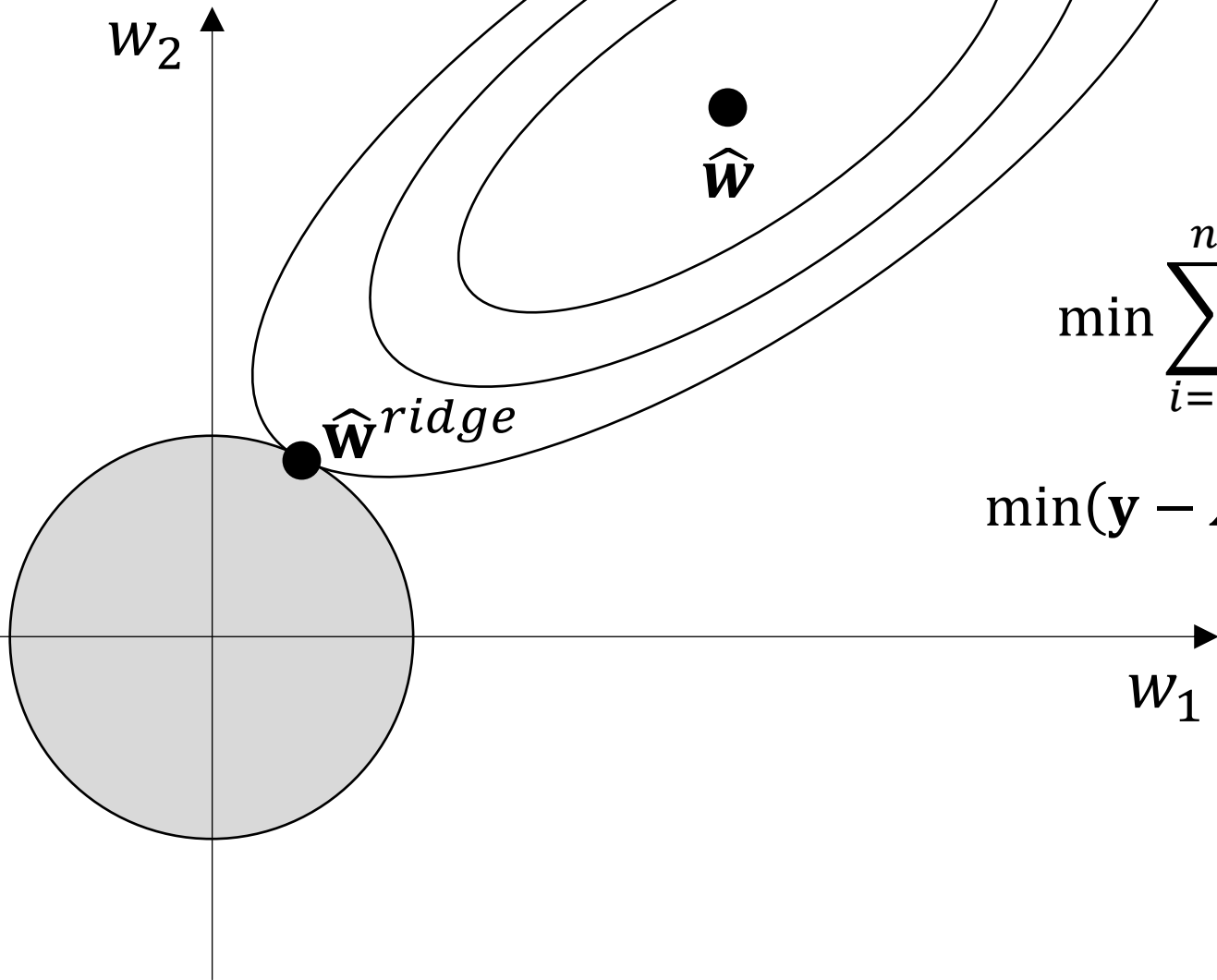


Ridge Regression

L2 정규화
(L2 regularization)



릿지 회귀 분석



$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2,$$

$$s.t. \sum_{i=1}^p w_i^2 \leq t$$

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}),$$

$$s.t. \|\mathbf{w}\|_2^2 \leq t$$

릿지 회귀 분석 - 라그랑주 프리멀 함수로 표현

$$L_p = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\|\mathbf{w}\|_2^2 - t)$$

$$\hat{\mathbf{w}}^{ridge} = \underset{\mathbf{w}}{\operatorname{argmin}} \left\{ (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\|\mathbf{w}\|_2^2 - t) \right\}$$

릿지 회귀 분석

$$\frac{\partial L_p}{\partial \mathbf{w}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0$$

$$\Leftrightarrow -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\mathbf{w} + 2\lambda\mathbf{w} = 0$$

$$\Leftrightarrow (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}_P)\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\hat{\mathbf{w}}^{ridge} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}_P)^{-1}\mathbf{X}^T\mathbf{y}$$

람다의 역할

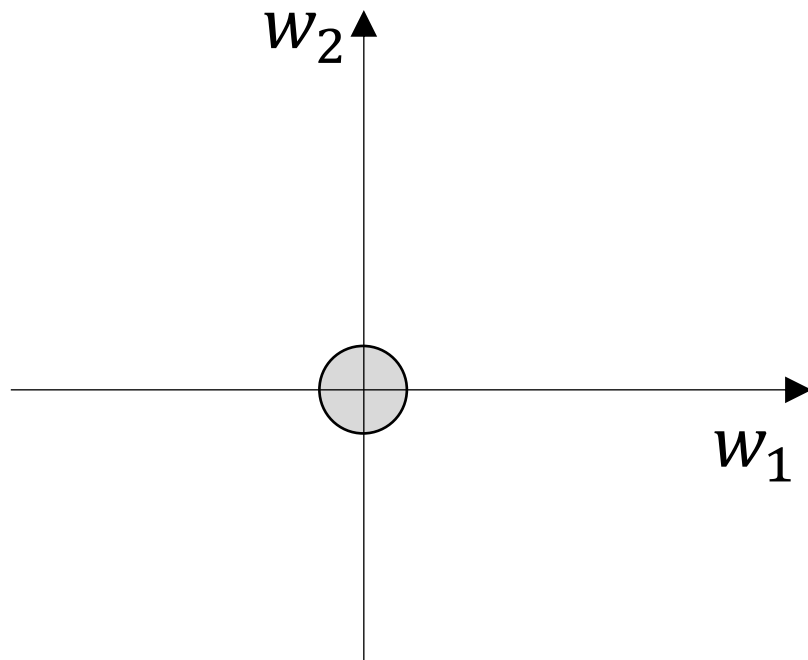
- λ 는 계수(coefficient)의 사이즈를 조절
- λ 는 정규식의 크기(amount)를 조절
- λ 가 0에 가까워질수록, 최소 제곱 추정량(least squared estimator)에 가까워짐
- λ 가 무한대에 가까워질수록 릿지 해는 0에 가까워집니다. 즉, 상수항(intercept)만 남은 모형에 가까워짐

제약식 가중치

$$\lambda \sum_{i=1}^p w_i^2 \leq t \Leftrightarrow \sum_{i=1}^p w_i^2 \leq \frac{t}{\lambda}$$

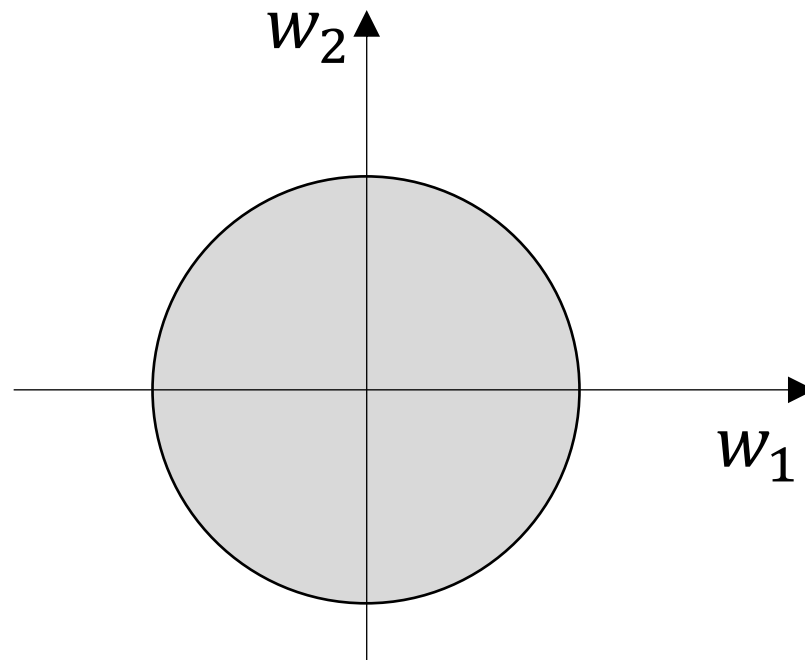
람다가 커지면?

-> 도형이 작아짐 -> 제약식이 강해짐

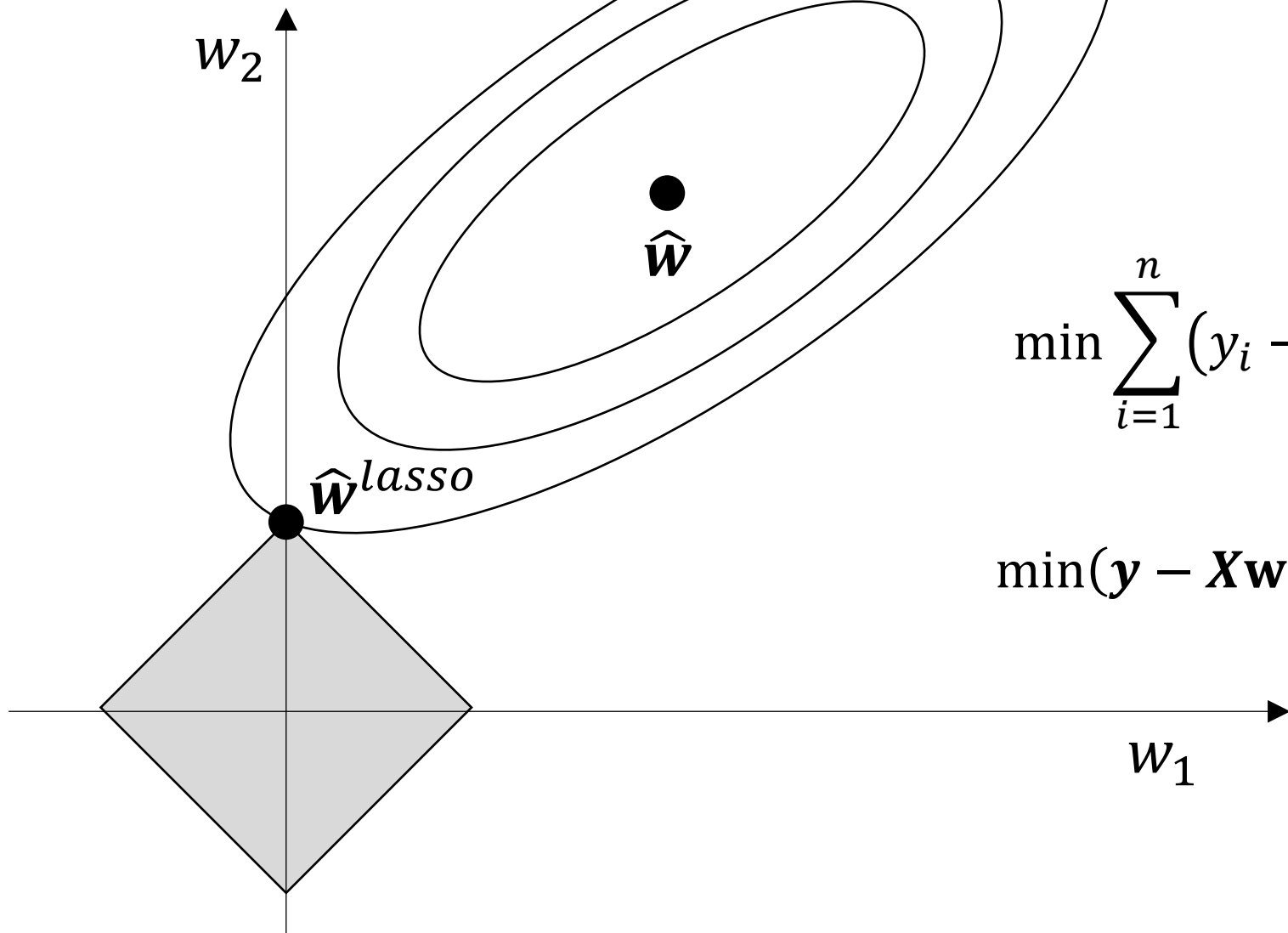


람다가 작아지면?

-> 도형이 커짐 -> 제약식이 약해짐



라쏘 회귀 분석



$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d |\mathbf{w}_i| \leq t$$

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad s.t. |\mathbf{w}| \leq t$$

라쏘 회귀 분석 - 라그랑주 프리멀 함수로 표현

$$L_P = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(|\mathbf{w}| - t)$$

$$\hat{\mathbf{w}}^{lasso} = \underset{\mathbf{w}}{\operatorname{argmin}} \{ (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(|\mathbf{w}| - t) \}$$

라쏘 회귀 분석 - 추정량

라쏘 추정량은 확실한 형태(closed form solution)가 존재하지 않음

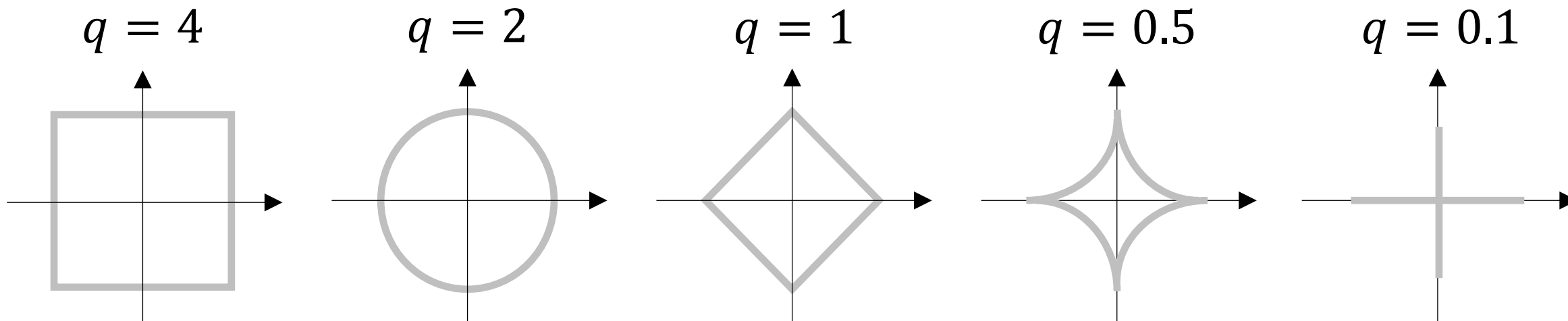
라쏘 추정량은 closed form이 존재하지 않기 때문에,

소프트웨어 기반으로 경사하강법과 같은 numerical optimization을 사용하여 구함

제약식의 일반화된 표현

$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d |\mathbf{w}_i|^q \leq t$$

$$\tilde{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \{ (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda (|\mathbf{w}|^q - t) \}$$



엘라스틱 넷(Elastic net)

엘라스틱 넷은 릿지 회귀 분석과 라쏘 회귀 분석을 합쳐놓은 형태
L1 제약식과 L2 제약식을 동시에 적용

$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d (\alpha |w_i| + (1 - \alpha) w_i^2) \leq t$$

$$\min (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}), \quad s.t. (\alpha \|\mathbf{w}\|_1 + (1 - \alpha) \|\mathbf{w}\|_2^2) \leq t$$

엘라스틱 넷(Elastic net)

$$L_p = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\alpha|\mathbf{w}| + (1 - \alpha)\|\mathbf{w}\|_2^2 - t)$$

$$\hat{\mathbf{w}}^{elasticnet} = \underset{\mathbf{w}}{\operatorname{argmin}}\{(\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\alpha|\mathbf{w}| + (1 - \alpha)\|\mathbf{w}\|_2^2 - t)\}$$

AI/BigData 인텐시브 과정

Section 3. 선형 회귀 분석

Section 3-2. 선형 회귀 분석의 실습

선형 회귀 분석

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

선형 회귀 분석

```
In [5]: ▶ 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']  
2  
3 X = df[features]  
4 y = df['MEDV']
```

```
In [6]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [7]: ▶ 1 #데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```

선형 회귀 분석

In [8]: ▶

```
1 # 선형 회귀분석 학습
2 from sklearn.linear_model import LinearRegression
3 clf_lr = LinearRegression()
4 clf_lr.fit(X_tn_std, y_tn)
```

Out[8]:

```
▼ LinearRegression
LinearRegression()
```

In [9]: ▶

```
1 # 선형 회귀분석 모형 추정 계수 확인
2 print(clf_lr.coef_)
3 print(clf_lr.intercept_)
```

```
[-0.97100092  1.04667838 -0.04044753  0.59408776 -1.80876877  2.60991991
 -0.19823317 -3.00216551  2.08021582 -1.93289037 -2.15743759  0.75199122
 -3.59027047]
22.608707124010557
```

선형 회귀 분석

In [9]:

```
1 # 릿지 회귀분석(L2 제약식 적용)
2 from sklearn.linear_model import Ridge
3 clf_ridge = Ridge(alpha=1)
4 clf_ridge.fit(X_tn_std, y_tn)
```

Out[9]:

```
▼ Ridge ⓘ ?
Ridge(alpha=1)
```

In [10]:

```
1 # 라쏘 회귀분석(L1 제약식 적용)
2 from sklearn.linear_model import Lasso
3 clf_lasso = Lasso(alpha=0.01)
4 clf_lasso.fit(X_tn_std, y_tn)
```

Out[10]:

```
▼ Lasso ⓘ ?
Lasso(alpha=0.01)
```

In [11]:

```
1 # 엘라스틱넷
2 from sklearn.linear_model import ElasticNet
3 clf_elastic = ElasticNet(alpha=0.01, l1_ratio=0.01)
4 clf_elastic.fit(X_tn_std, y_tn)
```

Out[11]:

```
▼ ElasticNet ⓘ ?
ElasticNet(alpha=0.01, l1_ratio=0.01)
```


선형 회귀 분석

In [12]: ▶

```
1 # 예측
2 pred_lr = clf_lr.predict(X_te_std)
3 pred_ridge = clf_ridge.predict(X_te_std)
4 pred_lasso = clf_lasso.predict(X_te_std)
5 pred_elastic = clf_elastic.predict(X_te_std)
```

In [13]: ▶

```
1 print(pred_lr)
```

```
[24.95233283 23.61699724 29.20588553 11.96070515 21.33362042 19.46954895
 20.42228421 21.52044058 18.98954101 19.950983    4.92468244 16.09694058
 16.93599574  5.33508402 39.84434398 32.33549843 22.32772572 36.54017819
 31.03300611 23.32172503 24.92086498 24.26106474 20.71504422 30.45072552
 22.45009234  9.87470006 17.70324412 17.974775    35.69932012 20.7940972
 18.10554174 17.68317865 19.71354713 23.79693873 29.06528958 19.23738284
 10.97815878 24.56199978 17.32913052 15.20340817 26.09337458 20.87706795]
```

선형 회귀 분석

In [14]: ▶

```
1 # 모형 평가-R제곱값
2 from sklearn.metrics import r2_score
3 print(r2_score(y_te, pred_lr))
4 print(r2_score(y_te, pred_ridge))
5 print(r2_score(y_te, pred_lasso))
6 print(r2_score(y_te, pred_elastic))
```

```
0.6354638433202133
0.6345884564889055
0.6343061000666704
0.6322273400977834
```

In [15]: ▶

```
1 # 모형 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 print(mean_squared_error(y_te, pred_lr))
4 print(mean_squared_error(y_te, pred_ridge))
5 print(mean_squared_error(y_te, pred_lasso))
6 print(mean_squared_error(y_te, pred_elastic))
```

```
29.782245092302325
29.853763334547605
29.876831576246808
30.046664219036874
```

AI/BigData 인텐시브 과정

Section 4. 로지스틱 회귀 분석

Section 4-1. 로지스틱 회귀 분석 개념

로지스틱 회귀 분석

	선형 회귀분석	로지스틱 회귀분석
타겟 형태	연속형	범주형, 연속형
타겟 범위	제한 없음	제한 있음

로지스틱 회귀 분석

$$z = wx + b$$



기울기



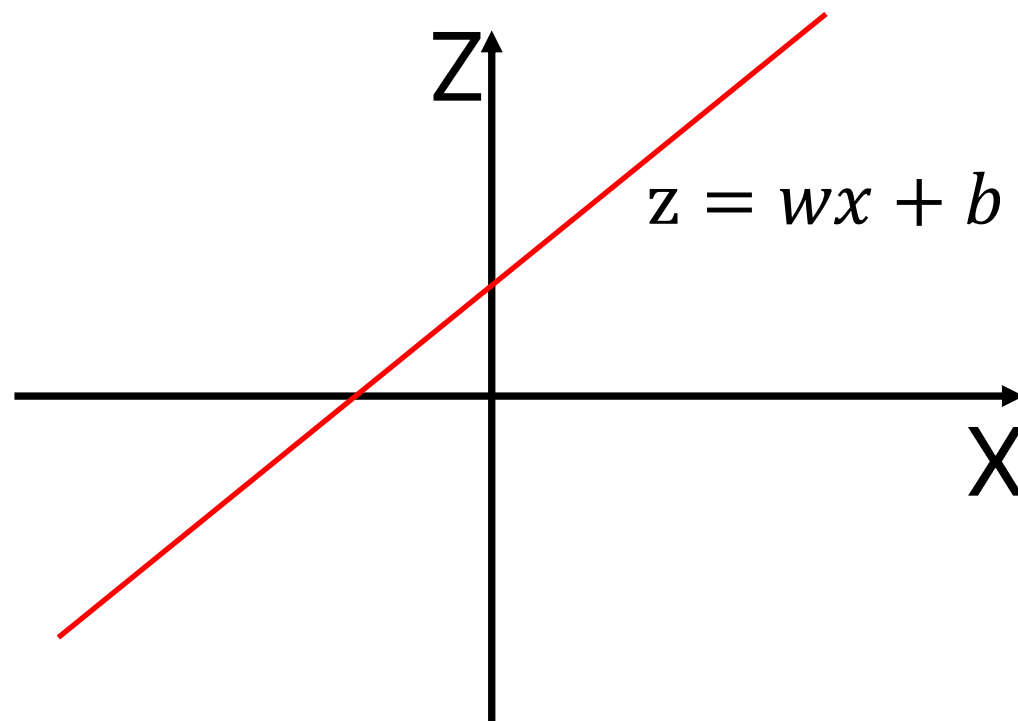
z 절편



$$-\infty < z < \infty$$



z에 제한이 있도록 변형 필요



로지스틱 회귀 분석

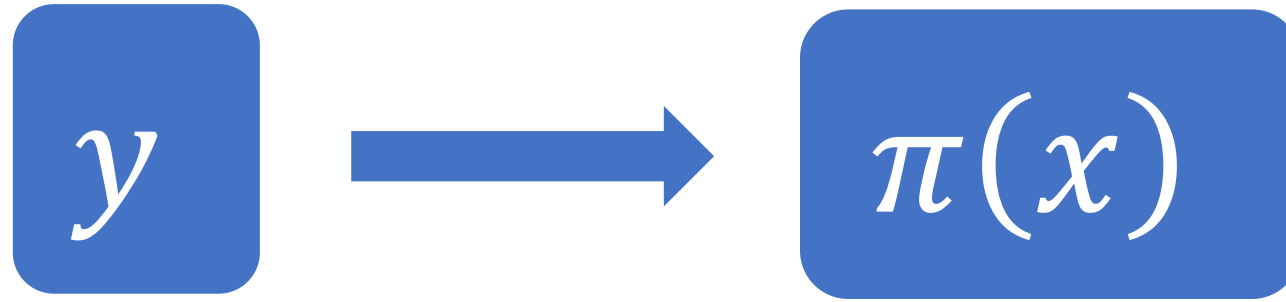
$$y = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(wx+b)}}$$

z의 변형을 통해 만들어진 새로운 변수

시그모이드 함수

$$\log \left(\frac{y}{1-y} \right) = wx + b$$

로지스틱 회귀 분석



$$\pi(x) = p(Y = 1 \mid X = x)$$

X=x 일때, Y=1일 확률

로지스틱 회귀 분석

$$\text{logit}(\pi(x)) = \log \left(\frac{\pi(x)}{1 - \pi(x)} \right) = wx + b$$

오즈비(odds ratio) = 양성 확률은 음성 확률의 몇배인가?

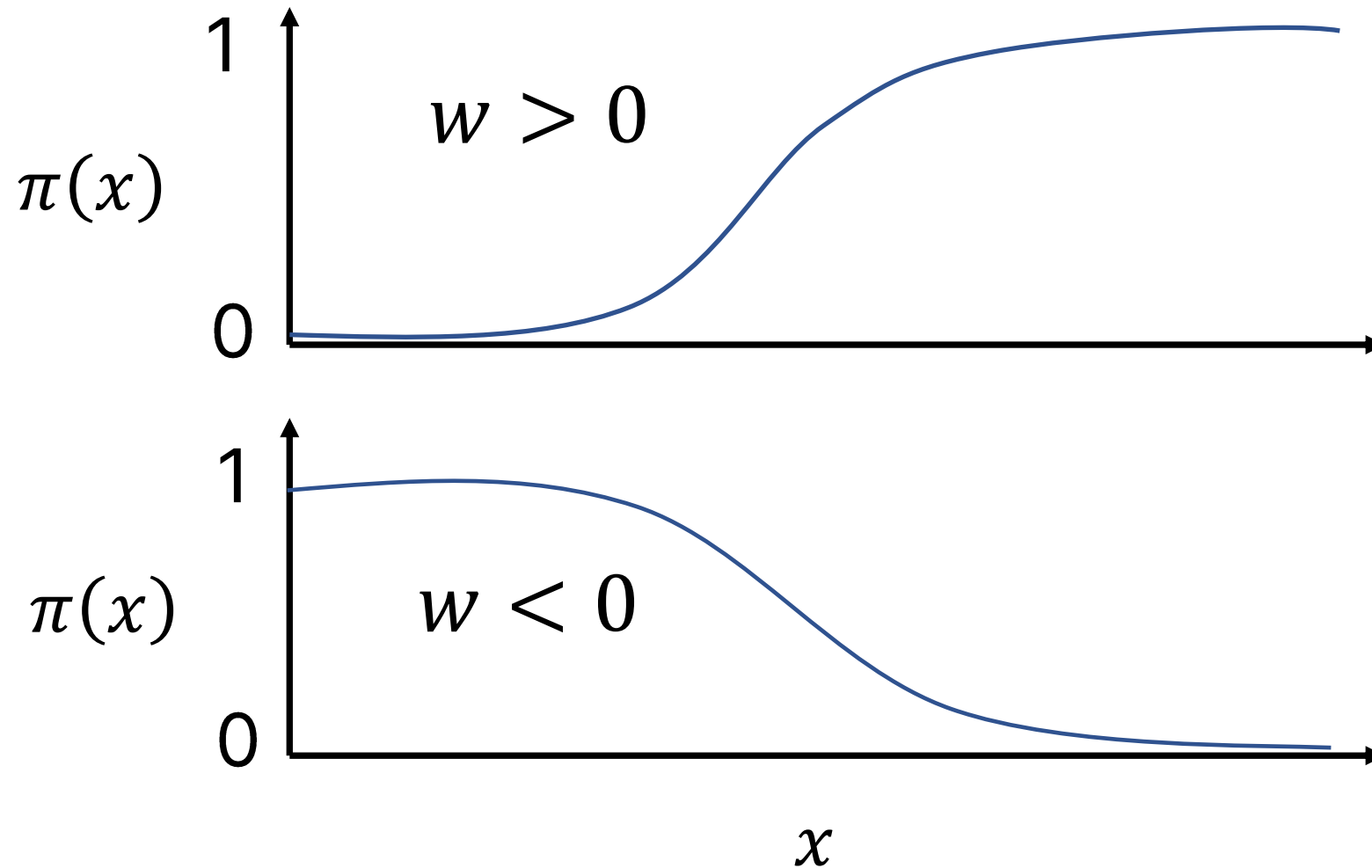
$$\text{logit}(\pi(x)) = \log \left(\frac{P(Y = 1 | X = x)}{P(Y = 0 | X = x)} \right) = wx + b$$

로지스틱 회귀 분석

$$\pi(x) = p(Y = 1 \mid X = x) = \frac{e^{wx+b}}{1+e^{wx+b}}$$

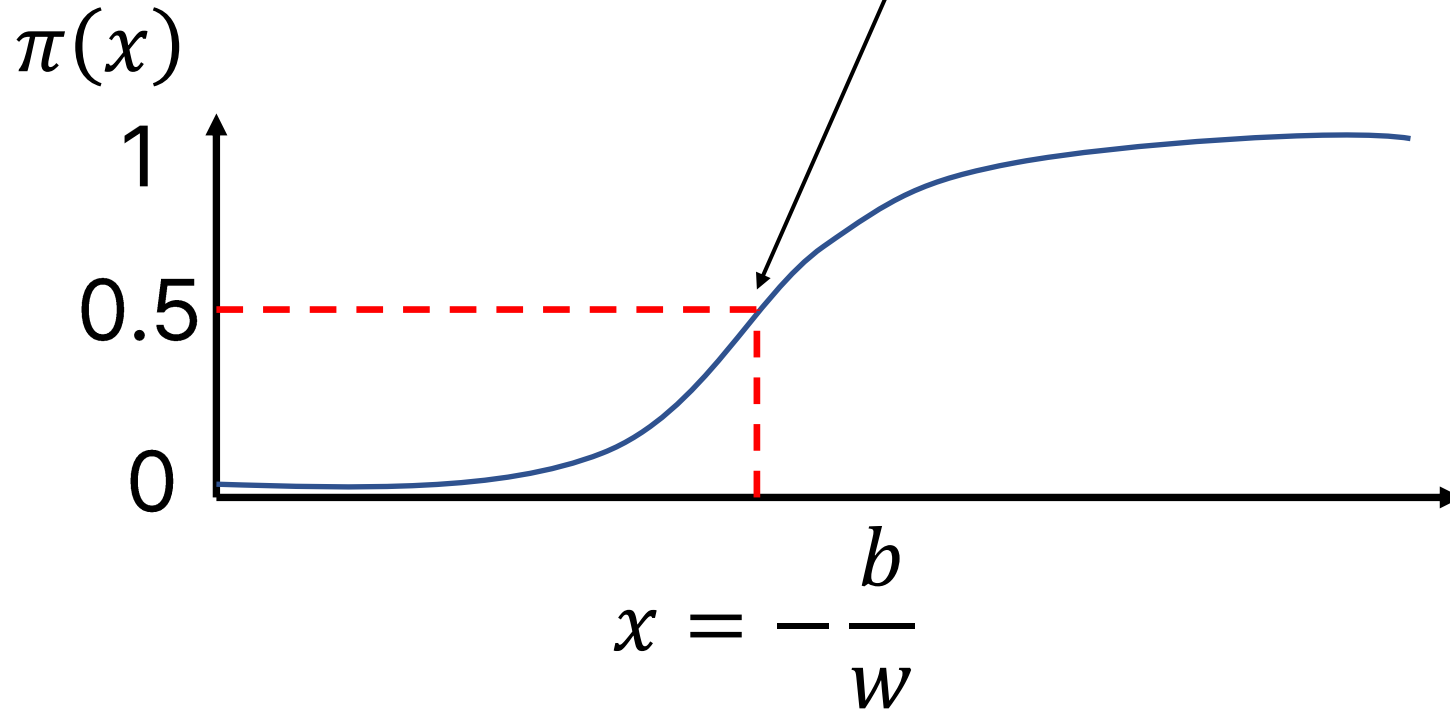
X=x 일때, Y=1일 확률

로지스틱 회귀 분석



로지스틱 회귀 분석

기울기가 가장 가파른 곳



기울기가 크다?



x가 조금만 증가해도 1일 확률이 급격히 증가함.

로지스틱 회귀 분석에서 w 를 구하는 과정

x_1	x_2	y
1	2	0

초기 가중치

$$\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \mathbf{w} = \begin{pmatrix} 0.5 \\ -0.3 \end{pmatrix}$$

$$b = 0.1$$

$$z = \mathbf{w}^T \mathbf{x} + b = (0.5 \quad -0.3) \begin{pmatrix} 1 \\ 2 \end{pmatrix} + 0.1 = 0$$

$$y = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} = \frac{1}{1 + e^0} = 0.5$$

y=1 일 확률 0.5

y	$\hat{y}$
0	0.5

loss 계산 후 그래디언트 디센트 방법으로 가중치 업데이트

AI/BigData 인텐시브 과정

Section 4. 로지스틱 회귀 분석

Section 4-1. 로지스틱 회귀 분석 실습

로지스틱 회귀 분석

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

로지스틱 회귀 분석

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

로지스틱 회귀 분석

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```


로지스틱 회귀 분석

```
In [6]: ▶ 1 # 로지스틱 회귀분석 (L2 제약식 적용)
          2 from sklearn.linear_model import LogisticRegression
          3 clf_logi_l2 = LogisticRegression(penalty='l2')
          4 clf_logi_l2.fit(X_tn_std, y_tn)
```

```
Out[6]: ▾ LogisticRegression
         LogisticRegression()
```

```
In [7]: ▶ 1 # 로지스틱 회귀분석 모형 (L2 제약식 적용) 추정 계수
          2 print(clf_logi_l2.coef_)
          3 print(clf_logi_l2.intercept_)

[[ 0.69853977  0.25867826  0.39868195 -0.76815236 -0.00429211  0.18526119
  0.74915884 -0.12654939  0.10401364  0.17318889  0.13724279  0.68065574
  0.98907933]
 [-0.81533159 -0.47295284 -0.74058812  0.54948684 -0.08525548  0.01376966
  0.18741135  0.13017416  0.324744   -1.07084932  0.57591149  0.1276171
 -0.99183488]
 [ 0.11679181  0.21427458  0.34190617  0.21866552  0.08954759 -0.19903085
 -0.93657019 -0.00362477 -0.42875763  0.89766042 -0.71315428 -0.80827284
  0.00275554]]
[ 0.29606777  0.57687138 -0.87293914]
```

로지스틱 회귀 분석

In [8]: ▶

```
1 # 예측
2 pred_logistic = clf_logi_l2.predict(X_te_std)
3 print(pred_logistic)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

In [9]: ▶

```
1 # 확률값으로 예측
2 pred_proba = clf_logi_l2.predict_proba(X_te_std)
3 print(pred_proba)
```

```
[[9.93360060e-01 4.92359807e-03 1.71634194e-03]
 [3.22076323e-04 2.77943948e-04 9.99399980e-01]
 [1.00535859e-02 9.89854907e-01 9.15071461e-05]
 [9.85578241e-01 1.15830075e-02 2.83875127e-03]
 [9.72825954e-02 8.94560998e-01 8.15640637e-03]
 [1.60778757e-01 8.37805552e-01 1.41569030e-03]
 [9.99185889e-01 2.82861666e-04 5.31249532e-04]]
```

로지스틱 회귀 분석

In [10]: ▶

```
1 # 정확도 (accuracy)
2 from sklearn.metrics import accuracy_score
3 accuracy = accuracy_score(y_te, pred_logistic)
4 print(accuracy)
```

1.0

In [11]: ▶

```
1 # 정밀도 (precision)
2 from sklearn.metrics import precision_score
3 precision = precision_score(y_te, pred_logistic, average='macro')
4 print(precision)
```

1.0

In [12]: ▶

```
1 # 리콜
2 from sklearn.metrics import recall_score
3 recall = recall_score(y_te, pred_logistic, average='macro')
4 print(recall)
```

1.0

In [13]: ▶

```
1 # F1 스코어
2 from sklearn.metrics import f1_score
3 f1 = f1_score(y_te, pred_logistic, average='macro')
4 print(f1)
```

1.0

로지스틱 회귀 분석

In [14]: ▶

```
1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_logistic)
4 print(conf_matrix)
```

```
[[16  0  0]
 [ 0 21  0]
 [ 0  0  8]]
```

In [15]: ▶

```
1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_logistic)
4 print(class_report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	1.00	1.00	1.00	21
2	1.00	1.00	1.00	8
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

감사합니다.

Q & A